TECHNICAL SPECIFICATION

V3.4 · 2026-04-24

LSIE-MLF

Technical Specification

Engineering Blueprint for Autonomous AI Implementation and Enterprise Deployment

Live Stream Inference Engine — Machine Learning Framework

CONTENT VERSION	SCHEMA VERSION	CLASSIFICATION	HARDENING BASELINE
v3.4	v3.0	CONFIDENTIAL	31-Item Autonomous Audit

CONTENTS

Table of contents

0 **Document control and canonical terminology** 3
- 0.1 Document purpose 3
- 0.2 Normative language 3
- 0.3 Canonical terminology registry 3
- 0.4 Version history 5
- 0.5 Specification amendments 6

1 **System philosophy and operational scope** 8

2 **Deterministic system data flow pipeline** 9

3 **Codebase architecture and directory mapping** 12
- 3.1 Directory hierarchy 12
- 3.2 Build context rules 13

4 **Core architecture modules** 13
- 4.A Hardware & transport 13
- 4.B Ground truth ingestion 15
- 4.C Orchestration & synchronization 17
- 4.D Multimodal ML processing 20
- 4.E Experimentation & analytics 22
- 4.F Context enrichment 24

5 **Data governance and security** 25
- 5.1 Ephemeral vault specification 25
- 5.2 Data classification 26

6 **Interface contracts and payload schemas** 27
- 6.1 InferenceHandoffPayload 27
- 6.2 PhysiologicalSampleEvent 34
- 6.3 PhysiologicalContext 36
- 6.4 BanditDecisionSnapshot 38
- 6.5 AttributionEvent 40
- 6.6 OutcomeEvent 44
- 6.7 EventOutcomeLink 45
- 6.8 AttributionScore 46

7 **Mathematical specifications** 48
- 7A AU12 facial action unit geometry 48
- 7B Thompson sampling reward pipeline 51

7C Physiological co-modulation analytics . . . 54
7D Observational acoustic analytics . . . 58
7E Event→Outcome attribution policy . . . 61

8 Deterministic semantic scorer specification 65
8.1 Inference parameter constraints . . . 66
8.2 Deterministic semantic scorer registry . . . 67
8.3 Expected output schema . . . 67
8.4 System prompt (canonical) . . . 68

9 Docker compose runtime topology 68
9.1 Container topology . . . 68
9.2 Volume mounts . . . 69
9.3 GPU allocation . . . 70
9.4 Device passthrough . . . 70
9.5 Network configuration . . . 70
9.6 Service dependency order . . . 70

10 Dependency and runtime matrix 71
10.1 System requirements . . . 71
10.2 Python package pinning . . . 71

11 Variable extraction matrix 72

12 Error handling and recovery specification 79
12.1 Network disconnection . . . 79
12.2 Hardware device loss . . . 80
12.3 Worker process crash . . . 80
12.4 Queue overload . . . 80

13 Autonomous implementation audit checklist 81

SECTION 0

Document control and canonical terminology

0.1 Document purpose

This document is the single authoritative specification for the Live Stream Inference Engine — Machine Learning Framework (LSIE-MLF). A developer or autonomous AI agent is required to build the complete LSIE-MLF system using only this document. No external documentation is assumed or required.

0.2 Normative language

Throughout this specification, the following terms carry precise normative meaning:

TERM	MEANING
MUST / SHALL / IS REQUIRED TO	The requirement is absolute and non-negotiable.
MUST NOT / SHALL NOT	The prohibition is absolute.
IS DEFINED AS	The term has a single, unambiguous meaning as stated.

0.3 Canonical terminology registry

Every system component has exactly one canonical name. All prior synonyms are retired. Implementations must use only the canonical identifiers listed below.

CANONICAL NAME	RETIRED SYNONYMS	DEFINITION
API Server	`api`, `FastAPI server`, `web server`, `ASGI server`	The FastAPI application process that serves REST endpoints on port 8000. Container name: api.
ML Worker	`worker`, `Celery node`, `Celery worker`, `GPU worker`, `inference worker`	The Celery consumer process that executes all GPU-bound ML inference tasks. Container name: worker.
Message Broker	`Redis`, `queue`, `task queue`, `broker`	The Redis in-memory data store that brokers Celery task queues between the API Server and the ML Worker. Container name: redis.
Persistent Store	`postgres`, `PostgreSQL`, `database`, `relational database`, `DB`	The PostgreSQL relational database that persists analytical metrics and experiment state. Container name: postgres.
Capture Container	`stream_scrcpy`, `scrcpy container`, `capture service`, `stream ingester`	The privileged Docker container responsible for physical USB I/O and raw audio-visual stream extraction. Container name: stream_scrcpy.

CANONICAL NAME	RETIRED SYNONYMS	DEFINITION
IPC Pipe	`FIFO, named pipe, POSIX pipe, audio pipe, kernel pipe, video pipe`	POSIX Named Pipes located at `/tmp/ipc/audio_stream.raw` (audio) and `/tmp/ipc/video_stream.mkv` (video) used for zero-copy data transfer between the Capture Container and the Orchestrator Container.
Ephemeral Vault	`24-hour vault, data vault, transient storage, secure buffer`	The AES-256-GCM encrypted transient storage regime for raw media buffers, subject to mandatory 24-hour secure deletion.
InferenceHandoffPayload	`handoff schema, payload schema, inference payload`	The JSON Schema Draft 07 contract governing data exchange between Module C and Module D, and from Module D to Module E.
Orchestrator Container	`orchestrator, segment producer, Module C worker`	Dedicated Docker container running the Module C orchestration loop. Uses the same image as the ML Worker but with a different CMD (`python3.11 -m services.worker.run_orchestrator`). Produces Celery tasks consumed by the ML Worker.
PhysiologicalChunkEvent	`Physiological Chunk Event, Physiological Sample Event, oura event, HRV event, wearable event, physio event`	A normalized wearable telemetry chunk record produced by the API Server hydration path and enqueued to Redis for the Orchestrator Container to drain. Defined by the PhysiologicalChunkEvent schema in §6.2.
Physiological Context	`physio context, HRV context, biometric context`	A per-subject derived snapshot of physiological state attached to each 30-second InferenceHandoffPayload segment via the _physiological_context dict key. The snapshot may include RMSSD, heart rate, validity_ratio, is_valid, freshness, staleness, source_kind, and derivation_method. Defined by the PhysiologicalContext schema in §6.3.
Physiological State Buffer	`physio buffer, HRV buffer, state cache`	An in-memory rolling buffer in the Orchestrator Container keyed by subject_role ('streamer' or 'operator'). Updated by draining PhysiologicalChunkEvent records from the physio:events Redis list and used to derive scalar physiological snapshots at segment assembly time.
Co-Modulation Index	`co-mod score, synchrony index, physiological correlation`	The Pearson correlation coefficient between 1-minute resampled, time-aligned valid streamer and operator RMSSD sequences over a rolling 10-minute analysis window. Computed by Module E and persisted to the comodulation_log table. Null when fewer than 4 paired observations are available or either aligned series has zero variance.
BanditDecisionSnapshot	`bandit snapshot, decision snapshot, selection snapshot`	The deterministic pre-update Thompson Sampling decision record captured by Module C at arm-selection time and attached to InferenceHandoffPayload segments for downstream attribution and offline evaluation.

CANONICAL NAME	RETIRED SYNONYMS	DEFINITION
AttributionEvent	`attribution event, event ledger row, encounter attribution record`	The canonical derived record persisted by Module E representing a stimulus-linked interaction event eligible for downstream event→outcome attribution under §7E.
OutcomeEvent	`conversion event, terminal event, outcome row`	The canonical derived record persisted by Module E representing a delayed downstream outcome such as creator_follow for event→outcome attribution under §7E.
EventOutcomeLink	`attribution link, event link, causal link row`	The canonical derived record persisted by Module E representing deterministic eligibility linkage between an AttributionEvent and an OutcomeEvent under a versioned horizon/linking rule.
AttributionScore	`attribution metric, score row, ledger score`	The canonical derived record persisted by Module E representing a method-specific attribution or observational score attached to an event or event→outcome pair.

0.4 Version history

VERSION	DATE	DESCRIPTION
v1.0	2026-03-12	Initial unified specification.
v2.0	2026-03-12	Production-hardened. Applied 15-rule deterministic audit: canonical terminology, module contracts, data-flow pipeline, dependency pinning, AU12 Python fix, IPC lifecycle, temporal sync formulas, Ephemeral Vault hardening, Docker topology expansion, LLM determinism constraints, error-handling matrices, variable matrix expansion, and autonomous implementation audit.
v3.0	2026-04-01	Mathematical recipe (Thompson Sampling, continuous reward, fractional Beta-Bernoulli update), E2E gap fixes, hardening baseline, 6 spec amendments incorporated (cuDNN 8, Ubuntu 24.04, orchestrator container, scrcpy v3.3.4, audio chunk 1/30s, INT8 locked), 63 new tests, reward pipeline formalization.
v3.1	2026-04-15	Physiological co-modulation extension. Added Oura Ring webhook ingestion (§4.B.2), Orchestrator physiological state buffer (§4.C.4), Module E co-modulation analytics (§4.E.2), PhysiologicalChunkEvent and PhysiologicalContext schemas (§6.2, §6.3), RMSSD co-modulation math (§7C), physiological data governance, failure matrix entries, variable matrix entries, and audit checklist items. §7B Thompson Sampling reward pipeline explicitly unchanged.
v3.2	2026-04-17	Physiological ingestion model corrected. Oura webhook handling no longer assumes provider-delivered scalar 5-minute RMSSD payloads. Webhooks are treated as change notifications that trigger OAuth2-backed hydration, normalized physiological chunk transport, rolling RMSSD derivation in Module C, validity gating, and 1-minute co-modulation resampling. Operator dashboard stack migrated from Streamlit to dual PySide6 host surfaces: a retained single-file developer debug tool and a modular production operator console. §7B reward pipeline remains unchanged.

VERSION	DATE	DESCRIPTION
v3.3	2026-04-23	Observational acoustic analytics extension. Added stimulus-windowed acoustic analysis in Module D, new mathematical recipe (§7D), canonical Module D → Module E payload fields, Persistent Store schema-oriented acoustic field additions, deterministic acoustic data-quality handling, and new audit checklist items. §7B Thompson Sampling reward pipeline remains unchanged.
v3.4	2026-04-24	Attribution analytics extension. Added Module E Attribution Analytics (§4.E.3), stable segment identity and BanditDecisionSnapshot carriage in InferenceHandoffPayload (§6.1), attribution schemas (§6.4), Event→Outcome Attribution Policy (§7E), deterministic semantic scorer registry with local cross_encoder primary and optional llm_gray_band fallback (§8), bounded semantic reason codes, variable matrix additions, and new audit controls. §7B Thompson Sampling reward pipeline remains unchanged.

0.5 Specification amendments

AMENDMENT	REF	ACCEPTED CHANGE
`SPEC-AMEND-001` 2026-03-20	9.1	Original: Worker image: `nvidia/cuda:12.2.2-cudnn9-runtime-ubuntu22.04` Amended: Worker image: `nvidia/cuda:12.2.2-cudnn8-runtime-ubuntu22.04` Rationale: cuDNN 9 lacks SM 6.1 (Pascal) binaries. Target hardware (GTX 1080 Ti) requires cuDNN 8 for dp4a INT8 vectorization. Incorporated into v3.0 spec text.
`SPEC-AMEND-002` 2026-03-20	9.1	Original: Capture Container image: `ubuntu:22.04` Amended: Capture Container image: `ubuntu:24.04` Rationale: scrcpy v3.1+ prebuilt Linux x86_64 binaries require GLIBC 2.38+, unavailable in Ubuntu 22.04 (ships GLIBC 2.35). Ubuntu 24.04 ships GLIBC 2.39. Incorporated into v3.0 spec text.
`SPEC-AMEND-003` 2026-03-20	9.1	Original: 5 containers (redis, postgres, stream_scrcpy, worker, api). Module C runs inside the worker process. Amended: 6 containers. Orchestrator runs as a separate container using the same image as the worker but with a different CMD (`python3.11 -m services.worker.run_orchestrator`). Rationale: The orchestrator is the PRODUCER of Celery tasks while the worker is the CONSUMER. Separating them prevents the Celery consumer's concurrency model from conflicting with the orchestrator's asyncio event loop and persistent FFmpeg/PyAV subprocesses. Incorporated into v3.0 spec text.
`SPEC-AMEND-004` 2026-03-20	4.A.1	Original: scrcpy v2.x with single-instance multiplexed audio/video capture. Amended: scrcpy v3.3.4 with dual-instance architecture: audio on port range 27100:27199 and video on port range 27200:27299. Direct `--record` to pipe replaces dd/fd3 for video path. Rationale: Dual-instance design eliminates cross-stream coupling and allows either scrcpy process to be restarted independently. Direct `--record` to MKV streaming container avoids fragile dd byte piping. Incorporated into v3.0 spec text.

AMENDMENT	REF	ACCEPTED CHANGE
SPEC-AMEND-005 2026-03-20	4.C.3	Original: Audio chunk size: 1 second (32000 bytes). Amended: Audio chunk size: 1/30 second (~1067 bytes) via `int(bytes_per_second / 30)`. Rationale: Aligns audio consumption cadence with 30 FPS video frame rate for temporal synchronization. Incorporated into v3.0 spec text.
SPEC-AMEND-006 2026-03-20	4.D.1	Original: TranscriptionEngine `compute_type` configurable at deployment time. Amended: TranscriptionEngine fixes class constant `_COMPUTE_TYPE` to `"int8"`. Not exposed as a deployment-time option. Rationale: Downstream effect of SPEC-AMEND-001: INT8 uses dp4a which is available on Pascal SM 6.1, FP16 is not. Locking the value prevents accidental misconfiguration. Incorporated into v3.0 spec text.
SPEC-AMEND-007 2026-04-15	§4.B.2, §4.C.4, §4.E.2, §6.2, §6.3, §7C, §5, §11, §12, §13	Original: No physiological telemetry ingestion, no wearable data integration, no co-modulation analytics. Amended: v3.1 adds physiological co-modulation extension: Oura Ring webhook ingestion via API Server (§4.B.2), Physiological State Buffer in Orchestrator (§4.C.4), optional _physiological_context on InferenceHandoffPayload segments, Module E co-modulation analytics (§4.E.2), RMSSD Pearson correlation math (§7C), physiology_log and comodulation_log database tables, and associated data governance, failure handling, and audit items. §7B reward pipeline unchanged. Rationale: Behavioral markers (AU12) are subject to performative masking. Integrating autonomic nervous system data (RMSSD via Oura Ring) provides a non-deceptive physiological signal for evaluating operator-streamer interaction quality. Deployed as observational analytics in v3.1; reward integration deferred to a future revision.
SPEC-AMEND-008 2026-04-17	§4.E.1, §10.2	Original: Operational metrics and experiment state are visualized through a Streamlit dashboard. Dependencies list includes Streamlit. Amended: Streamlit dashboard retired. Operator visualization is delivered through two PySide6 surfaces running on the operator host: (a) scripts/debug_studio.py retained as a single-file developer diagnostic GUI; (b) services/operator_console/ implemented as a modular production operator dashboard. All operator-surface data access flows through the FastAPI server. No direct Postgres or Redis coupling is added to the operator host. requirements/worker.txt removes streamlit. requirements/cli.txt pins PySide6. Rationale: Streamlit's rerun-oriented web model is not suitable for stateful operator workflows that require live video and metric readback, device-level operator actions, and long-running native sessions. PySide6 provides persistent, native host-side operator surfaces while keeping developer diagnostics separate from the production console.

AMENDMENT	REF	ACCEPTED CHANGE
SPEC-AMEND-009 2026-04-17	§4.B.2, §4.C.4, §6.2, §6.3, §11.3, §7C	Original: Oura webhook path assumes the provider delivers aggregated 5-minute RMSSD and heart-rate values directly into the webhook transport, and Module C stores only the latest scalar snapshot. Amended: Physiological ingestion model revised. Oura Ring webhook deliveries are treated as change notifications rather than authoritative scalar RMSSD samples. The API Server verifies, deduplicates, subject-maps, and enqueues hydration work. OAuth2-backed hydration fetches provider resources and normalizes them into PhysiologicalChunkEvent records for Orchestrator consumption. Module C maintains rolling per-subject physiological buffers, derives scalar RMSSD snapshots over a trailing 5-minute window, computes validity_ratio, applies a strict validity gate, and attaches derived _physiological_context snapshots to segment payloads. Module E computes co-modulation on 1-minute re-sampled, time-aligned valid RMSSD snapshots over a 10-minute window. §7B reward pipeline unchanged. Rationale: The prior scalar-webhook assumption is structurally incorrect for daytime Oura ingestion. The corrected design preserves the existing API → Redis → Orchestrator boundary, keeps physiological persistence scalar-only, and makes the co-modulation math internally consistent with the 10-minute analysis window.
SPEC-AMEND-010 2026-04-24	§4.C.2, §4.E.3, §6.1, §6.4, §7E, §8, §11, §13	Original: No first-class attribution ledger, no stable segment_id in InferenceHandoffPayload, semantic evaluation specified only as deterministic Azure OpenAI free-form reasoning output, and no event→outcome attribution schemas. Amended: v3.4 adds attribution analytics as an observational extension: stable segment_id and BanditDecisionSnapshot in InferenceHandoffPayload; Module E Attribution Analytics (§4.E.3); AttributionEvent, OutcomeEvent, EventOutcomeLink, AttributionScore, and BanditDecisionSnapshot schemas (§6.4); Event→Outcome Attribution Policy (§7E); deterministic semantic scorer registry with local cross_encoder primary and optional llm_gray_band fallback; bounded semantic reason codes; and new attribution audit controls. §7B reward pipeline unchanged. Rationale: The system requires first-class event identity, future off-policy-evaluation-enabling decision capture, and a lower-latency deterministic semantic chokepoint improvement without violating the audited reward path.

SECTION 1

System philosophy and operational scope

LSIE-MLF ingests USB-tethered multimodal streams—PCM audio, H.264 video, and chat metadata—and runs concurrent ML pipelines to produce structured metrics. Thompson Sampling uses AU12 smile intensity as a dense proxy reward for the sparse creator-follow event.

The architecture is schema-first and deterministic. USB media transport, temporal synchronization, message passing, and ML inference must behave predictably across independent audio, video, and chat clocks.

The system applies strict data minimization. Raw biometric media does not persist beyond the transient pipeline; persistent storage is limited to derived, anonymized outputs under the Ephemeral Vault policy (§5).

SECTION 2

Deterministic system data flow pipeline

This section defines the complete end-to-end data path through the LSIE-MLF system. Every transition specifies the transport mechanism, payload format, synchronization mechanism, and error behavior.

STAGE 1 · MOBILE HARDWARE → CAPTURE CONTAINER

TRANSPORT	USB passthrough via `/dev/bus/usb` device mapping into the Capture Container; scrcpy v3.3.4 runs as two coordinated instances, with an audio process (`--no-video` plus `--record` to the audio IPC Pipe) on port range 27100:27199 and a video process (`--no-audio` plus `--record` to the video IPC Pipe) on port range 27200:27299.
PAYLOAD	Audio: raw PCM signed 16-bit little-endian (s16le), 48 kHz mono written to `/tmp/ipc/audio_stream.raw`. Video: H.264 carried in an MKV streaming container written to `/tmp/ipc/video_stream.mkv`.
SYNC	Android hardware clock polled via `adb shell` 'echo `$EPOCHREALTIME`' every 30 seconds; drift offset applied to timestamps.
ERROR BEHAVIOR	If USB connection is lost, the Capture Container logs the event, halts pipe writes, and polls reconnection every 2 seconds for 60 seconds before graceful termination.

STAGE 2 · CAPTURE CONTAINER → ORCHESTRATOR CONTAINER

TRANSPORT	Audio IPC Pipe at `/tmp/ipc/audio_stream.raw` via the shared Docker volume ipc-share, consumed by the Orchestrator Container's FFmpeg AudioResampler subprocess. In parallel, the video IPC Pipe `/tmp/ipc/video_stream.mkv` is consumed by PyAV through VideoCapture in the same Orchestrator Container.
PAYLOAD	Audio: continuous byte stream of PCM s16le at 48 kHz. Video: H.264 carried in an MKV streaming container.
SYNC	The audio FIFO is held open with `exec 3<> /tmp/ipc/audio_stream.raw`, while the video FIFO remains unshielded so a broken PyAV reader forces a fresh MKV header on restart.
ERROR BEHAVIOR	If the Orchestrator Container audio reader crashes, pipe writes return EAGAIN once the buffer fills. Overflowed bytes are silently discarded until the consumer restarts.

STAGE 3 · ORCHESTRATOR CONTAINER INTERNAL: AUDIO RESAMPLING

TRANSPORT	AudioResampler launches the `FFMPEG_RESAMPLE_CMD` subprocess inside the Orchestrator Container to convert `/tmp/ipc/audio_stream.raw` from 48 kHz mono s16le into 16 kHz mono s16le on `pipe:1`.
PAYLOAD	Input: PCM s16le 48 kHz. Output: PCM s16le 16 kHz via `pipe:1`.
SYNC	FFmpeg executes synchronously with the audio processing thread ensuring ordered delivery.
ERROR BEHAVIOR	If FFmpeg exits unexpectedly, the orchestrator detects the failure and restarts the AudioResampler subprocess within 1 second.

STAGE 4 · MODULE B → MODULE C

TRANSPORT	Event dictionaries appended to the orchestrator deque buffer for inclusion in the next segment window.
PAYLOAD	Python dictionaries containing `uniqueId`, `event_type`, `timestamp_utc`, and payload fields.
SYNC	Events timestamped using host UTC and drift-corrected offset before entering the deque ring buffer.
ERROR BEHAVIOR	If the WebSocket disconnects, exponential backoff reconnection is attempted (1s initial delay, 30s max, 10 retries).

STAGE 5 · ORCHESTRATOR CONTAINER → ML WORKER

TRANSPORT	Celery asynchronous task dispatch via `process_segment.delay(payload)` from the Orchestrator Container, through the Redis Message Broker, to the ML Worker.
PAYLOAD	InferenceHandoffPayload JSON object validated against the Pydantic schema, including `session_id`, deterministic `segment_id`, stable segment window bounds, experiment attribution fields, AU12 telemetry, optional `_physiological_context`, and `_bandit_decision_snapshot`.
SYNC	The Orchestrator Container assembles fixed 30-second segments, computes deterministic `segment_window_start_utc`, `segment_window_end_utc`, and `segment_id` values, captures a pre-update bandit decision snapshot at arm-selection time, validates the payload, and dispatches it asynchronously before inference begins in the ML Worker.
ERROR BEHAVIOR	If schema validation fails, the segment is logged and discarded while processing continues.

STAGE 6 · MODULE D → MODULE E

TRANSPORT	Celery asynchronous task dispatch through the Message Broker.
PAYLOAD	JSON payload containing `session_id`, `segment_id`, stable `segment_window_start_utc` and `segment_window_end_utc`, carried-forward `_bandit_decision_snapshot`, optional carried-forward `_physiological_context`, Module C-derived AU12 telemetry from `_au12_series`, transcription, semantic outputs (`reasoning` bounded reason code, `is_match`, `confidence_score`, `semantic_method`, and `semantic_method_version`), and the canonical §7D observational acoustic analytics fields. Legacy segment-level scalar fields `pitch`, `jitter`, and `shimmer` are deprecated as canonical outputs in this version; implementations may dual-write legacy and canonical acoustic fields for one compatibility version, but downstream consumers must migrate to the canonical windowed fields.
SYNC	Redis append-only persistence is enabled in the Message Broker configuration so queued tasks survive broker restarts.
ERROR BEHAVIOR	If the broker is unreachable Celery retries with exponential backoff up to five attempts before writing failed payloads to `/data/processed/failed_tasks/`.

STAGE 7 · MODULE E → PERSISTENT STORE

TRANSPORT	psycopg2-binary PostgreSQL connection pool.
PAYLOAD	`SQL INSERT` statements with parameterized queries storing numeric metrics as `DOUBLE PRECISION`, validity flags as `BOOLEAN`, and timestamps as `TIMESTAMPTZ`.
SYNC	SERIALIZABLE isolation for experiment updates and READ COMMITTED for metric inserts.
ERROR BEHAVIOR	If the database is unreachable the module buffers up to 1000 records in memory and retries every 5 seconds before overflowing to CSV.

STAGE 8 · API SERVER → ORCHESTRATOR CONTAINER

TRANSPORT	Normalized physiological chunk events are pushed to the Redis list `physio:events` with `RPUSH` by the API Server hydration layer and drained with non-blocking `LPOP` by the Orchestrator Container. The drain is bounded to 100 events per cycle so physiological traffic cannot stall the 30-second segment dispatch cadence.
PAYLOAD	JSON dictionaries matching PhysiologicalChunkEvent, containing `unique_id`, `event_type`, `provider`, `subject_role`, `source_kind`, `window_start_utc`, `window_end_utc`, `ingest_timestamp_utc`, and nested payload fields `ibi_ms_items`, `rmssd_items_ms`, `heart_rate_items_bpm`, `motion_items`, `sample_interval_s`, `valid_sample_count`, `expected_sample_count`, and `derivation_method`.
SYNC	Provider timestamps remain in UTC and freshness is computed at segment assembly time from wall-clock age relative to `source_timestamp_utc` derived from chunk `window_end_utc`. The ADB DriftCorrector is not applied because physiological events do not originate from the Android device clock.
ERROR BEHAVIOR	If signature verification or schema validation fails, the API Server rejects the webhook deterministically (401 or 422). If Redis is unavailable at webhook-ingress time it returns 503 so the provider retries. Hydration failures after successful enqueue are retried asynchronously. Stale physiological snapshots are annotated with `is_stale = true`; they do not block segment dispatch, and Module E excludes stale snapshots from co-modulation computation.

SECTION 3

Codebase architecture and directory mapping

The repository uses a segmented monorepo: each computational domain is isolated by directory but integrated through a root Docker Compose topology. Shared utilities, math libraries, and interface contracts are available without private package registries.

API logic and ML logic are physically separated because the API Server runs an asynchronous FastAPI/Uvicorn event loop while the ML Worker runs GPU-bound Celery tasks. This preserves non-blocking behavior and improves Docker build caching.

3.1 Directory hierarchy

PATH	PURPOSE
`/docker-compose.yml`	The root orchestrator defining the multi-container topology, virtual networks, GPU device reservations, and shared IPC volume mounts. This file is the single entry point for all container lifecycle operations.
`/services/api/`	Houses the API Server logic. Contains the ASGI entry point (Uvicorn), RESTful routing controllers, and dependency injection modules for the Persistent Store connection pool.
`/services/operator_console/`	Modular PySide6 production operator console. Runs on the operator host and accesses system data only through the FastAPI server.
`/services/worker/`	Contains the shared worker image code, including ML Worker Celery task consumers and the Module C orchestrator entry point. Runtime separation is provided by distinct Docker Compose services with different commands.
`/services/stream_ingest/`	Dedicated to the Capture Container's raw audio-visual extraction mechanisms. Contains the shell orchestrators responsible for instantiating the IPC Pipe and executing the ADB screen mirroring and audio extraction protocols.
`/packages/ml_core/`	Houses the shared mathematical and inference utilities utilized across the stack. This includes the Python reference implementations for topological facial landmark extraction, Action Unit mathematics, audio transcription wrappers.
`/packages/schemas/`	Centralizes Pydantic models and JSON Schema Draft 07 definitions ensuring strict interface contracts and data validation rules across services.
`/data/raw/`	Local host volume mount for transient staging of extracted audio-visual buffers including IPC pipe buffers and raw PCM streams. Mounted as tmpfs in production.
`/data/interim/`	Storage for intermediate processing states such as downsampled audio buffers and cropped video frames for localized neural network analysis.

PATH	PURPOSE
`/data/processed/`	Destination for validated ML inference outputs and structured analytical JSON payloads prior to database ingestion.
`/scripts/`	Host-side developer utilities, including the single-file PySide6 diagnostic tool scripts/debug_studio.py.
`/requirements/`	Maintains deterministic dependency tracking separating base requirements, API dependencies, and heavy ML dependencies for optimized Docker layer caching.
`/data/sql/`	PostgreSQL DDL and seed data files. Mounted read-only to `docker-entrypoint-initdb.d/` for automatic schema initialization on first container startup. Contains experiment definition inserts and metrics table schemas.
`/services/worker/pipeline/`	Module C orchestration and Module E analytics pipeline. Houses the orchestrator, reward computation, and video capture pipeline modules used by the orchestrator and worker containers.

3.2 Build context rules

All Dockerfiles under `/services` must use the monorepo root as build context so shared `/packages` code is available during build. `.dockerignore` rules must keep unrelated assets out of images. The API Server image excludes heavy ML dependencies; worker images exclude unnecessary web assets.

SECTION 4

Core architecture modules

The modules defined below form the operational spine of the LSIE-MLF specification. Each module contains explicit execution details followed by a formal contract defining inputs, outputs, dependencies, side effects, and failure modes.

4.A Hardware & transport

Purpose. Extract low-latency audio and video streams from tethered mobile hardware and transport them into the containerized environment without REST or broker serialization overhead.

Technology stack. scrcpy v3.3.4, dual-instance scrcpy architecture (audio on port range 27100:27199 and video on port range 27200:27299), IPC Pipe (POSIX Named Pipe) transport for separate audio and video streams, Bash file descriptor manipulation (fd 3 audio shield), Docker shared volume ipc-share

4.A.1 EXECUTION DETAILS

Module A maps the host `/dev/bus/usb` subsystem into the Capture Container and starts two coordinated scrcpy v3.3.4 processes against the same Android device. One process captures audio and the other captures video.

The dual-instance design keeps the transport path simple and restartable. Audio is written directly to `/tmp/ipc/audio_stream.raw` through scrcpy's `--record` path, while video is written directly to `/tmp/ipc/video_stream.mkv`. Splitting the streams avoids cross-stream coupling and allows either scrcpy process to be relaunched independently when only one media path fails.

The capture design uses intentionally asymmetric pipe lifecycle handling so audio continuity and video container recovery each fail in the intended way under independent scrcpy restarts.

Startup is staggered by 4 seconds to prevent concurrent ADB server pushes to the device. Before every capture cycle, the container clears `adb forward` and `adb reverse` tunnels, then launches the audio instance first and the video instance second. Both instances use `--no-playback` and write into the shared ipc-share volume instead of routing media through REST endpoints or Redis payloads.

- Audio scrcpy flags: `--no-video`, `--no-playback`, `--audio-codec=raw`, `--audio-buffer=30`, `--audio-dup`, `--record=$AUDIO_PIPE`, `--port=27100:27199`, `--tunnel-host=$HOST_IP`.
- Video scrcpy flags: `--no-audio`, `--no-playback`, `--video-codec=h264`, `--max-fps=30`, `--record=$VIDEO_PIPE`, `--record-format=mkv`, `--port=27200:27299`, `--tunnel-host=$HOST_IP`.

> **VIDEO FIFO INTENTIONALLY LEFT UNSHIELDED**
>
> Do not symmetrize the pipe handling. The shell keeps only the audio FIFO open with `exec 3<> $AUDIO_PIPE`; the video FIFO stays unshielded so a PyAV-side failure breaks the write path, forces video scrcpy to exit, and lets the next launch emit a fresh MKV header.

IPC PIPE LIFECYCLE SPECIFICATION

1. **Directory creation:** The Capture Container initializes the shared IPC mount by running `mkdir -p /tmp/ipc` before any scrcpy process is started.
2. **Pipe cleanup and creation:** `setup_pipes()` removes stale FIFO files with `rm -f` and recreates both `/tmp/ipc/audio_stream.raw` and `/tmp/ipc/video_stream.mkv` with `mkfifo`.
3. **Audio fd 3 shield:** The shell opens `exec 3<> $AUDIO_PIPE` so the audio pipe stays open across reader or scrcpy restarts and the writer is decoupled from consumer timing.
4. **Video pipe left unshielded:** No extra descriptor is opened for `/tmp/ipc/video_stream.mkv`; if the PyAV consumer drops the pipe, the video scrcpy process exits and the next launch emits a fresh MKV header.
5. **Device wait gate:** `wait_for_device()` polls `adb devices` every 2 seconds for up to 60 seconds and terminates the container if no Android device reports a ready device state.
6. **ADB tunnel cleanup:** Each capture loop starts by running `adb forward --remove-all` and `adb reverse --remove-all` so stale tunnels cannot collide with the audio and video port ranges.
7. **Audio stream launch:** The audio scrcpy instance launches first and writes directly to `/tmp/ipc/audio_stream.raw` with `--record` while binding port range 27100:27199 through `$HOST_IP`.

8. **Staggered video launch:** After a 4-second delay, the video scrcpy instance launches and writes H.264 in an MKV streaming container directly to `/tmp/ipc/video_stream.mkv` on port range 27200:27299.
9. **Monitoring and restart:** The container waits on both scrcpy PIDs with `wait -n`; when either process exits, both children are cleaned up, the device is rechecked, and the dual-stream capture loop restarts.
10. **Cross-launch cleanup:** On the next container start, `setup_pipes()` again runs `rm -f` on both FIFO paths before recreating them, preventing stale shared-volume pipe files from surviving across launches.

MODULE A — FORMAL CONTRACT **§ 4.A.contract**

INPUTS	USB-connected Android device producing raw PCM audio (s16le 48 kHz) and H.264 encoded video via scrcpy.
OUTPUTS	Continuous PCM audio stream (s16le, 48 kHz, mono) written to `/tmp/ipc/audio_stream.raw`, and continuous H.264 video in an MKV streaming container written to `/tmp/ipc/video_stream.mkv`; both are exposed as IPC Pipes on the shared Docker volume ipc-share.
DEPENDENCIES	scrcpy v3.3.4, ADB server via `ADB_SERVER_SOCKET=tcp:host.docker.internal:5037`, `/dev/bus/usb` device passthrough, Docker volume ipc-share mounted at `/tmp/ipc/`, environment variables `SDL_VIDEODRIVER=dummy`, `XDG_RUNTIME_DIR=/tmp`, `SCRCPY_SERVER_PATH`, and `SCRCPY_ICON_PATH`, plus dedicated scrcpy port ranges 27100:27199 (audio) and 27200:27299 (video).
SIDE EFFECTS	Creates two named pipes (`/tmp/ipc/audio_stream.raw` and `/tmp/ipc/video_stream.mkv`), opens file descriptor 3 as an audio-pipe shield, spawns persistent audio and video scrcpy processes, and establishes ADB forward/reverse tunnels for port ranges 27100:27199 and 27200:27299.
FAILURE MODES	1. USB disconnect causes wait_for_device() to poll every 2 seconds for up to 60 seconds before the container exits. 2. If the audio scrcpy process crashes, wait -n detects the exit, both scrcpy PIDs are cleaned up, and the loop restarts while fd 3 keeps /tmp/ipc/audio_stream.raw open. 3. If the video path breaks, the unshielded /tmp/ipc/video_stream.mkv pipe lets the video scrcpy process fail so the next launch writes a fresh MKV header for PyAV. 4. Each capture cycle removes adb forward/reverse tunnels to prevent port collisions, and if the downstream consumer lags, pipe writes may return EAGAIN so overflowed data is silently discarded per §12.

4.B Ground truth ingestion

Purpose. Ingest real-time contextual metadata, chat streams, and user interactions to create ground-truth datasets.

Technology stack. Python 3.11.x, TikTokLive connector, EulerStream signature API

4.B.1 EXECUTION DETAILS

The system continuously ingests live chat and interaction events from the platform using the TikTokLive Python connector.

Connection requires ByteDance anti-crawler signatures including `X-Bogus` and `msToken`. Signature generation is delegated to the EulerStream signature server.

Once connected the ingestion module enforces the `Action_Combo` constraint where multiple events must coincide temporally to form a valid ground truth signal.

Incoming protobuf messages are parsed into Python dictionaries and high-value composite events are staged for later synchronization with inference outputs.

4.B.2 PHYSIOLOGICAL INGESTION ADAPTER

Physiological webhook ingestion is implemented as a FastAPI POST route in the API Server container. It is separate from TikTok WebSocket ingestion and has its own authentication, validation, and transport rules.

Oura webhook deliveries trigger an OAuth2-backed hydration step that fetches the updated provider resource and normalizes it into physiological chunk records.

- Verify `x-oura-signature` with `OURA_WEBHOOK_SECRET`, resolve `subject_role`, and suppress duplicates with Redis `SETNX physio:seen:{unique_id}`.
- Parse only the notification metadata needed to route hydration.
- Enqueue hydration work on `physio:hydrate`; the hydrator refreshes tokens as needed and emits normalized PhysiologicalChunkEvent records using `source_kind` ibi or session.
- Append normalized chunk events to `physio:events` with `RPUSH` for Orchestrator consumption.
- The route emits explicit HTTP failures for signature, schema, and Redis-availability problems.

OURA WEBHOOKS ARE CHANGE NOTIFICATIONS, NOT RMSSD SAMPLES

Treat Oura Ring webhooks as change notifications only. Do not read them as authoritative `rmssd_ms` samples or derive RMSSD from generic heartrate notifications; verify, deduplicate, hydrate the provider resource, and normalize the result into PhysiologicalChunkEvent records.

RAW WEBHOOK/PROVIDER PAYLOADS MUST NEVER PERSIST

Keep raw webhook JSON and hydrated provider response bodies transient. They must not be persisted; only normalized downstream records may continue, with `401` on signature failure, `422` on schema failure, and `503` when Redis is unavailable.

INPUTS	Authenticated TikTok Webcast WebSocket event stream using EulerStream-signed URL parameters. Additionally, authenticated physiological webhook payloads arrive via the API Server and are enqueued to a Redis list for downstream consumption (§4.B.2).
OUTPUTS	Parsed dictionaries representing high-value TikTok events containing `uniqueId`, `event_type`, `timestamp_utc`, `gift_value`, and `combo_events`; normalized PhysiologicalChunkEvent records appended to Redis list `physio:events`; and hydration work items queued on `physio:hydrate`.
DEPENDENCIES	TikTokLive library, EulerStream signature API, network access to TikTok Webcast servers, FastAPI route handling, Redis client, Oura webhook secret, Oura OAuth2 hydration client and token storage, and schema validation for PhysiologicalChunkEvent.
SIDE EFFECTS	Maintains persistent WebSocket connection and refreshes signature tokens during reconnects. Verifies physiological webhook signatures, deduplicates provider notifications, enqueues hydration work, and appends normalized physiological chunk records to Redis.
FAILURE MODES	1. WebSocket disconnection triggers exponential backoff reconnect. 2. EulerStream outage results in degraded mode. 3. Malformed protobuf events are logged and discarded. 4. Invalid physiological webhook signatures are rejected with 401. 5. Malformed physiological payloads are rejected with 422. 6. Redis unavailability returns 503 for provider retry. 7. OAuth2 hydration failures are retried asynchronously without fabricating physiological defaults.

4.C Orchestration & synchronization

Purpose. Module C runs as a dedicated Orchestrator Container built from the worker image and started with `python3.11 -m services.worker.run_orchestrator`. It polls ADB for drift correction, resamples 48 kHz PCM to 16 kHz through a persistent FFmpeg subprocess, processes video frames for AU12, assembles fixed 30-second segments, and dispatches InferenceHandoffPayload tasks to the Celery worker.

Technology stack. asyncio event loop, FFmpeg persistent resampling subprocess, PyAV-backed video pipe consumer, MediaPipe Face Mesh via packages/ml_core, ADB epoch polling, Celery task dispatch, Message Broker (Redis), psycopg2/PostgreSQL session registration and experiment lookup, collections.deque buffering

4.C.1 TEMPORAL DRIFT CORRECTION

DriftCorrector polls the Android epoch every 30 seconds by invoking ADB and computes the drift offset as host UTC minus Android epoch before segment timestamps are emitted.

Each corrected timestamp is produced by adding the cached drift offset to the original timestamp, and the orchestrator warns when absolute drift exceeds 150 milliseconds.

When ADB polling degrades, drift correction falls back to deterministic cached-offset behavior instead of introducing unstable timestamp jitter.

> **DRIFT CORRECTION FREEZES AFTER REPEATED ADB FAILURES**
>
> After three consecutive ADB poll failures, DriftCorrector freezes the last cached offset and keeps returning it while frozen. If the frozen state lasts 300 seconds, reset the offset to zero until a later successful poll refreshes it.

4.C.2 VIDEO TELEMETRY AND SEGMENT ASSEMBLY

The Orchestrator Container keeps a long-lived video capture thread on `/tmp/ipc/video_stream.mkv`, samples the latest frame inside the asyncio loop, and lazy-loads MediaPipe Face Mesh together with the AU12 normalizer on first use.

Before stimulus injection, incoming frames only accumulate the *B_neutral* calibration baseline. After the greeting trigger is recorded, the orchestrator stores a drift-corrected stimulus timestamp and appends per-frame AU12 observations `{timestamp_s, intensity}` to `_au12_series` for downstream reward computation.

In parallel, audio bytes accumulate until a full 30-second window is available. The Orchestrator Container then drains the event deque, computes deterministic `segment_window_start_utc`, `segment_window_end_utc`, and `segment_id` values, captures a pre-update BanditDecisionSnapshot at arm-selection time, assembles an InferenceHandoffPayload, attaches `_active_arm`, `_experiment_id`, `_expected_greeting`, `_stimulus_time`, `_au12_series`, `_x_max`, and `_bandit_decision_snapshot`, base64-encodes binary fields for Celery JSON transport, and dispatches the segment to Module D.

4.C.3 AUDIO RESAMPLING

AudioResampler launches a persistent FFmpeg subprocess inside the Orchestrator Container to convert the raw 48 kHz mono IPC stream at `/tmp/ipc/audio_stream.raw` into 16 kHz mono s16le PCM emitted on `pipe:1`.

The read loop runs at video cadence so audio consumption stays synchronized with 30 FPS segment assembly.

If the FFmpeg subprocess is not running, `read_chunk()` waits 1 second and starts a fresh resampler. EOF or pipe read errors stop the old process so the next iteration can recover automatically.

> **AUDIO CHUNKING IS 1/30 SECOND, NOT 1 SECOND**
>
> Consume audio in 1/30-second reads. With `bytes_per_second` = 32000, set `chunk_size` = `int(bytes_per_second / 30)` so each iteration reads 1,066 bytes (approximately 1,067), not a 1-second 32000-byte chunk.

4.C.4 PHYSIOLOGICAL STATE BUFFER

Module C keeps an in-memory Physiological State Buffer so 30-second inference segments can use wearable data that arrives asynchronously and at irregular cadence.

The Orchestrator drains chunk events from Redis, updates a rolling per-subject buffer, derives a current scalar snapshot over the trailing derivation window, and may attach the resulting physiological context to the outgoing segment payload.

- Store a time-ordered deque per `subject_role` covering at least `PHYSIO_BUFFER_RETENTION_S` seconds (default 900). Drain at most 100 events per cycle from `physio:events`.
- Derive each snapshot over `PHYSIO_DERIVE_WINDOW_S` seconds (default 300). Prefer overlapping ibi chunks; otherwise use overlapping session chunks; otherwise `rmssd_ms = null`.
- IBI-derived RMSSD uses `RMSSD = sqrt((1 / (N - 1)) * sum_{i=1}^{N-1}(IBI_{i+1} - IBI_i)^2)`. Heart rate is the rounded mean of overlapping `heart_rate_items_bpm` values or null.
- Compute `validity_ratio = sum(valid_sample_count) / max(sum(expected_sample_count), 1)` and set `is_valid = validity_ratio ⩾ PHYSIO_VALIDITY_MIN` (default 0.80). Invalid windows force `rmssd_ms = null`.
- Set `source_timestamp_utc` from the latest contributing `window_end_utc`, carry `source_kind` and `derivation_method`, and compute `freshness_s` from wall clock time without using ADB drift correction.
- Use `PHYSIO_STALENESS_THRESHOLD_S` (default 600) to classify snapshot age and surface staleness metadata on derived snapshots.

_PHYSIOLOGICAL_CONTEXT IS OPTIONAL

Omit `_physiological_context` entirely when no physiological snapshot exists. Do not emit empty objects, null-filled placeholders, or fabricated defaults.

STALE PHYSIOLOGY DOES NOT BLOCK DISPATCH

When `freshness_s` exceeds `PHYSIO_STALENESS_THRESHOLD_S`, mark `is_stale = True` and continue dispatch. Staleness annotates the payload; it does not block segment assembly or downstream inference.

DRIFT POLLING SPECIFICATION

PARAMETER	VALUE
Polling interval	30 seconds (DRIFT_POLL_INTERVAL)
ADB command	adb shell 'echo $EPOCHREALTIME'
Return format	Decimal epoch string with microsecond precision parsed as float
Maximum tolerated drift	150 milliseconds (MAX_TOLERATED_DRIFT_MS)
Drift formula	drift_offset = host_utc - android_epoch
Timestamp correction	corrected_ts = original_ts + drift_offset
Freeze threshold	3 consecutive ADB failures (DRIFT_FREEZE_AFTER_FAILURES)
Reset timeout	300 seconds / 5 minutes of frozen state (DRIFT_RESET_TIMEOUT)

FFMPEG RESAMPLING COMMAND

FFmpeg resampling command — BASH

```bash
ffmpeg -f s16le -ar 48000 -ac 1 -i /tmp/ipc/audio_stream.raw -ar 16000 -f s16le -ac 1 pipe:1
```

INPUTS	Raw PCM audio from `/tmp/ipc/audio_stream.raw`, H.264 video frames decoded from `/tmp/ipc/video_stream.mkv` by the PyAV-backed video capture thread, ground-truth event dictionaries drained from the in-memory deque, Android epoch timestamps returned by ADB drift polls, and operator stimulus trigger events used to mark greeting injection time.
OUTPUTS	16 kHz resampled mono PCM, drift-corrected segment timestamps, deterministic segment_id, stable segment window bounds in `segment_window_start_utc` and `segment_window_end_utc`, InferenceHandoffPayload segments dispatched as Celery tasks to the ML Worker, per-frame AU12 telemetry in `_au12_series`, stimulus timestamps in `_stimulus_time`, experiment attribution fields `_active_arm`, `_experiment_id`, `_expected_greeting`, `_x_max`, and a pre-update `_bandit_decision_snapshot` captured at arm-selection time.
DEPENDENCIES	FFmpeg persistent subprocess, PyAV-backed video capture, MediaPipe Face Mesh and AU12 normalization via `packages/ml_core`, ADB drift polling, Celery task dispatch, Message Broker (Redis), PostgreSQL access for session registration and Thompson Sampling arm selection, shared Docker volume ipc-share mounted at `/tmp/ipc/`, and data-raw, data-interim, and data-processed volumes.
SIDE EFFECTS	Spawns a persistent FFmpeg subprocess, starts and revives the video capture thread, registers the session in PostgreSQL, selects a Thompson Sampling arm and expected greeting, captures a deterministic pre-update BanditDecisionSnapshot at selection time, computes a stable segment_id from deterministic segment window identity, accumulates AU12 calibration and telemetry state in memory, and dispatches assembled segments to the Celery worker.
FAILURE MODES	1. ADB poll failures freeze the last drift offset after 3 consecutive errors and reset it to zero after 300 seconds of frozen state. 2. Missing or crashed FFmpeg subprocesses wait 1 second before automatic restart, while EOF and read errors stop the old process so the next iteration can recover. 3. Video capture thread death triggers a best-effort stop/recreate/start cycle in the run loop after pipe crashes, AU12 frame processing failures are debug-logged and skipped, event buffer overflow is bounded by deque(maxlen=10000) eviction, and Celery dispatch failures are logged while the loop continues.

4.D Multimodal ML processing

Purpose. Perform speech transcription, observational acoustic analysis, and deterministic semantic evaluation.

Technology stack. faster-whisper v1.2.1, praat-parselmouth v0.4.4, spacy v3.7.x, sentence-transformers v3.2.x, transformers v4.46.x

4.D.1 SPEECH TRANSCRIPTION

Speech transcription is executed by TranscriptionEngine, which wraps faster-whisper with the CTranslate2 backend on the pinned worker runtime.

For each segment, the Celery task stages the 16 kHz s16le PCM bytes through a temporary file and uses FFmpeg to prepare a transcribable audio container before calling `engine.transcribe()`.

GPU execution targets CUDA 12 with cuDNN 8, matching the worker image. cuDNN 8 is required to preserve Pascal SM 6.1 dp4a compatibility for INT8 inference on the target hardware (GTX 1080 Ti).

COMPUTE_TYPE IS FIXED TO INT8

`compute_type` is not a deployment-time knob. TranscriptionEngine fixes `_COMPUTE_TYPE` to `"int8"` to match the Pascal / cuDNN 8 inference path and prevent accidental misconfiguration.

WRAP RAW PCM IN WAV BEFORE TRANSCRIPTION

Before calling `engine.transcribe()`, wrap the temporary 16 kHz s16le PCM bytes in a WAV container with FFmpeg. Headerless PCM is not the intended direct ASR input.

4.D.2 LEGACY COMPUTER VISION COMPATIBILITY NOTE

Canonical AU12 extraction and MediaPipe Face Mesh inference are owned by Module C in v3.4 so per-frame telemetry can be attached to InferenceHandoffPayload before ML Worker dispatch.

Module D must not re-run computer-vision inference for the live reward path. It may consume already-derived AU12 telemetry carried on the handoff payload for validation, replay, or compatibility diagnostics, but any such use is observational only under the §7B reward-path invariance rule.

4.D.3 ACOUSTIC ANALYSIS

The Praat engine accessed through parselmouth extracts observational acoustic analytics from speech audio. Module D computes canonical acoustic summaries and deterministic null-gated validity metadata according to §7D.

For each segment with non-null `_stimulus_time`, Module D must derive stimulus-locked baseline and measurement windows using the same temporal boundaries defined in §7B, perform voiced-only masking and perturbation-span selection according to §7D, and emit the canonical §7D observational acoustic analytics fields to Module E.

If `_stimulus_time` is null, Module D must emit the false/null acoustic outputs defined in §7D.

This analytics path is observational only under the §7B reward-path invariance rule.

4.D.4 TEXT PRE-PROCESSING

spaCy performs tokenization, normalization, and linguistic preprocessing prior to deterministic semantic scoring.

In v3.4 the primary semantic scorer is a local Cross-Encoder executed in the ML Worker. Azure OpenAI is retained only as an optional deterministic gray-band fallback under §8 when the calibrated primary score falls inside a configured ambiguity interval.

INPUTS	16 kHz audio stream and InferenceHandoffPayload segments containing Module C-derived AU12 telemetry.
OUTPUTS	Speech transcripts, semantic evaluation JSON results containing `reasoning`, `is_match`, and `confidence_score` where `reasoning` is a bounded semantic reason code and `confidence_score` is the active method's semantic probability estimate, and the canonical §7D observational acoustic analytics fields. Reward computation continues downstream on the §7B path and this analytics path is observational only under the §7B reward-path invariance rule.
DEPENDENCIES	faster-whisper 1.2.1 with CTranslate2, praat-parselmouth 0.4.4, spaCy 3.7.*, sentence-transformers 3.2.*, transformers 4.46.*, PyTorch 2.4.*, optional Azure OpenAI fallback via the OpenAI Python SDK for gray-band semantic evaluation, FFmpeg for WAV container wrapping, and CUDA 12.2.2 runtime with cuDNN 8.
SIDE EFFECTS	Loads ML models into GPU memory, executes the primary local deterministic semantic scorer, may perform external HTTPS calls to Azure OpenAI only when gray-band fallback is enabled, and emits canonical observational acoustic analytics plus semantic outputs to Module E. This analytics path is observational only under the §7B reward-path invariance rule.
FAILURE MODES	1. GPU OOM triggers worker restart. 2. Whisper load failure aborts startup. 3. Local semantic scorer failure triggers deterministic fallback routing when `llm_gray_band` is enabled; if fallback is unavailable or times out, Module D records a bounded semantic error reason code and deterministic false/null-compatible outputs according to §8. 4. Insufficient voiced coverage, invalid perturbation spans, undefined Praat outputs, and local acoustic extraction exceptions are data-quality conditions; Module D catches them locally and emits deterministic validity flags and null dependent acoustic fields.

4.E Experimentation & analytics

Purpose. Module E aggregates inference outputs into persistent analytics, computes rewards, updates Thompson Sampling state, persists attribution analytics, and serves operator-facing analytics through FastAPI-backed PySide6 surfaces.

Technology stack. psycopg2-binary v2.9.x, pandas v2.2.x, SciPy, FastAPI analytics APIs, PySide6 operator surfaces (host)

4.E.1 EXECUTION DETAILS

Module E aggregates high-frequency inference outputs, reward analytics, physiological/co-modulation metrics, and observational acoustic analytics into structured time-series records.

Reward computation continues to follow §7B and returns a RewardResult dataclass from the established reward path.

Module E persists observational acoustic analytics emitted by Module D to the Persistent Store metrics path and may expose them through operator-facing analytics read APIs. This analytics path is observational only under the §7B reward-path invariance rule.

Operator visualization now runs on the operator host through two PySide6 surfaces, while all data access continues to flow through the FastAPI server.

- `scripts/debug_studio.py`: single-file developer diagnostic GUI for live video, landmarks, AU12 normalization, and analytics polling.
- `services/operator_console/`: modular production console for session monitoring, experiment readback, physiology/co-modulation views, and operator actions.

REWARD ENTRY POINT REMAINS COMPUTE_REWARD()

Physiological, observational acoustic, semantic-probability, and attribution extensions do not change §7B reward computation. `compute_reward()` remains the entry point and still consumes `_au12_series`, `_stimulus_time`, and `is_match`.

4.E.2 PHYSIOLOGICAL CO-MODULATION ANALYTICS

Module E computes rolling physiological co-modulation between the operator and the streamer using valid RMSSD snapshots derived in Module C.

A positive Pearson correlation indicates autonomic alignment and a negative value indicates divergence; the metric is stored as analytics output alongside other session measurements.

- Persist one physiological snapshot per subject per segment to `physiology_log`, including RMSSD, heart rate, validity, freshness, staleness, provider, `source_kind`, and `derivation_method`.
- Resample eligible RMSSD snapshots to 1-minute bins and compute Pearson correlation over a 10-minute rolling window.
- Coverage and variance checks determine whether a Pearson value can be emitted for a window.
- Persist `session_id`, window metadata, Pearson `r`, pair count, coverage ratio, and per-subject RMSSD means to `comodulation_log`.

CO-MODULATION IS OBSERVATIONAL ONLY

This analytics path is observational only under the §7B reward-path invariance rule.

NULL CO-MODULATION IS AN EXPECTED RESULT

A null co-modulation value is valid when there are fewer than 4 paired observations, zero variance in either aligned series, or insufficient eligible overlap. Record the null and continue; do not treat it as a pipeline error.

4.E.3 ATTRIBUTION ANALYTICS

Module E persists a derived attribution ledger that standardizes event identity, delayed outcome capture, deterministic event→outcome eligibility linking, and method-specific attribution scores.

Attribution analytics consume existing inference outputs, semantic evaluation outputs, AU12-derived reward diagnostics, physiological/co-modulation summaries, and delayed outcome events. This analytics path is observational only under the §7B reward-path invariance rule.

- Persist AttributionEvent, OutcomeEvent, EventOutcomeLink, and AttributionScore records.
- Persist only derived and versioned attribution artifacts; raw biometric media, raw physiological chunks, and free-form semantic rationales must not persist.
- Treat missing outcomes, closed attribution horizons, and null linkage as analytical states rather than worker failures.

> **ATTRIBUTION IS ANALYTICAL ONLY IN V3.4**
> Attribution analytics are observational only under the §7B reward-path invariance rule.

MODULE E — FORMAL CONTRACT **§ 4.E.contract**

INPUTS	Inference payloads from Module D delivered through the Celery task queue, plus delayed outcome records ingested through the analytics path for attribution finalization.
OUTPUTS	Time-series records stored in the Persistent Store, including canonical observational acoustic analytics, physiological/co-modulation analytics, AttributionEvent / OutcomeEvent / EventOutcomeLink / AttributionScore records, Thompson Sampling experiment assignments and posterior updates (α `+= r_t`, β `+= 1 − r_t`), RewardResult dataclass produced by `compute_reward()` (§7B), and operator-facing analytics and session-state read APIs consumed by PySide6 host surfaces.
DEPENDENCIES	psycopg2-binary, pandas, SciPy, Message Broker (Redis), Persistent Store (PostgreSQL).
SIDE EFFECTS	Writes metrics, canonical observational acoustic analytics, physiological/co-modulation results, attribution ledger records, and reward results to Persistent Store tables, updates Thompson Sampling Beta posterior parameters per encounter, and exposes analytics and experiment state through the FastAPI server for consumption by PySide6 operator surfaces.
FAILURE MODES	1. Persistent Store outage buffers 1000 records before disk overflow. 2. Celery task deserialization errors are logged and discarded.

4.F Context enrichment

Purpose. Perform asynchronous scraping of external web sources to enrich contextual information about the stream.

Technology stack. Celery v5.4.x, Message Broker, patchright v1.58.2

4.F.1 EXECUTION DETAILS

Context enrichment tasks run asynchronously outside the main inference loops.

Celery workers execute scraping jobs that gather external metadata about creators or trending topics.

patchright is used instead of standard Playwright to bypass bot-detection mechanisms by patching Chromium automation fingerprints.

MODULE F — FORMAL CONTRACT **§ 4.F.contract**

INPUTS	Celery tasks containing target URL, `scrape_type`, and `timeout_seconds` parameters.
OUTPUTS	JSON objects containing scraped metadata including `source_url`, `scraped_at_utc` timestamp, and extracted data dictionary.
DEPENDENCIES	Celery, Redis Message Broker, patchright browser automation toolkit, Chromium runtime.
SIDE EFFECTS	Launches ephemeral headless browser instances and performs outbound HTTP requests.
FAILURE MODES	1. CAPTCHA or HTTP blocking returns empty result. 2. Browser crash triggers retry. 3. Timeout terminates browser. 4. Broker outage triggers Celery reconnection.

SECTION 5

Data governance and security

LSIE-MLF processes sensitive speech and biometric data. The system therefore enforces strict data minimization, encryption, and retention controls so personally identifiable media does not persist beyond transient processing.

5.1 Ephemeral vault specification

PARAMETER	SPECIFICATION
Encryption algorithm	AES-256-GCM (authenticated encryption with associated data)
Encryption library	PyCryptodome (pycryptodome >= 3.20.0)
Key length	256 bits
IV/Nonce length	96 bits (12 bytes) generated using `os.urandom(12)`
Key generation	`os.urandom(32)` generating a unique key per debug session
Key storage location	Process memory only. Keys are never written to disk, environment variables, or configuration files.
Key lifecycle	Generated at session start and destroyed when the container terminates. No persistence mechanism exists.
Secure deletion method	`shred -vfz -n 3` executed by internal cron every 24 hours on `/data/raw/` and `/data/interim/`

PARAMETER	SPECIFICATION
Retention period	Maximum 24 hours for any raw media buffer touching physical disk

5.2 Data classification

Transient Data

Raw PCM audio buffers and decoded video frames existing only in volatile memory during processing. This tier also includes raw Oura Ring webhook JSON payloads and normalized PhysiologicalChunkEvent records while they remain in volatile queue transit or other transient memory-backed paths.

STORAGE	/tmp/ipc/ (tmpfs) and kernel pipe buffers and Redis list `physio:events`
RETENTION	Duration of active processing only with no disk persistence. Physiological webhook payloads must remain transient; if they ever touch disk for debugging, they inherit Ephemeral Vault controls and the 24-hour secure deletion cycle.

Debug Storage

Encrypted diagnostic media buffers written to disk only when algorithmic anomalies occur.

STORAGE	/data/raw/ and /data/interim/
RETENTION	Maximum 24 hours followed by secure deletion.

Permanent Analytical Storage

Anonymized analytical metrics including AU12 scores, scalar physiological metrics, co-modulation metrics, acoustic metrics, bounded semantic evaluation outputs, and attribution analytics.

STORAGE	Persistent Store (PostgreSQL)
RETENTION	Indefinite subject to database backup policies.

The system must not store raw facial images, voiceprints, raw Oura webhook JSON, hydrated provider response bodies, complete transient PhysiologicalChunkEvent payload bodies, or free-form semantic rationales in the Persistent Store. Persistent storage is limited to derived, normalized outputs such as AU12 metrics, scalar RMSSD, heart rate, bounded semantic reason codes, semantic probability estimates, co-modulation metrics, acoustic metrics, and attribution analytics.

SECTION 6

Interface contracts and payload schemas

Distributed multimodal pipelines require strict interface contracts. Incorrect array serialization or missing fields can break downstream indexing, alignment, and persistence.

LSIE-MLF uses JSON Schema Draft 07 and Pydantic validation to keep module boundaries deterministic at runtime.

The schemas below define the base media transport contract and the experiment attribution fields that the Orchestrator attaches before Celery dispatch. These fields carry arm selection, stimulus timing, AU12 telemetry, calibration state, stable segment identity, and BanditDecisionSnapshot capture for the reward pipeline (§7B) and attribution analytics (§7E).

Two physiological schemas are defined here. PhysiologicalChunkEvent (§6.2) describes the normalized chunk emitted by the API Server hydration path and consumed by the Orchestrator. PhysiologicalContext (§6.3) describes the optional `_physiological_context` snapshot attached to InferenceHandoffPayload.

v3.4 adds attribution schemas in §6.4 defining BanditDecisionSnapshot, AttributionEvent, OutcomeEvent, EventOutcomeLink, and AttributionScore.

Compatibility note: the JSON payload field `physiological_sample_event_schema` is retained for backward compatibility with existing renderers and Pydantic payload loaders. Its content is the canonical PhysiologicalChunkEvent schema. All prose and implementation identifiers must use PhysiologicalChunkEvent.

When `_physiological_context` is present, the PhysiologicalContext schema requires at least one real role snapshot. Empty objects, null-filled placeholders, and fabricated defaults must not validate or be emitted.

6.1 InferenceHandoffPayload

InferenceHandoffPayload JSON

```
{
 "$schema": "http://json-schema.org/draft-07/schema#",
 "title": "InferenceHandoffPayload",
 "description": "Standardized schema for multimodal ML pipeline handoff. v3.4 adds stable
 ↪  segment identity and pre-update bandit decision capture for attribution analytics while
 ↪  preserving \u00a77B reward semantics.",
 "type": "object",
 "required": [
  "session_id",
  "segment_id",
  "segment_window_start_utc",
  "segment_window_end_utc",
  "timestamp_utc",
  "media_source",
```

```
    "segments",
    "_active_arm",
    "_experiment_id",
    "_expected_greeting",
    "_stimulus_time",
    "_au12_series",
    "_x_max",
    "_bandit_decision_snapshot"
   ],
   "properties": {
    "session_id": {
     "type": "string",
     "pattern": "^[0-9a-f]{8}-[0-9a-f]{4}-4[0-9a-f]{3}-[89ab][0-9a-f]{3}-[0-9a-f]{12}$",
     "description": "UUID v4 representing the continuous live stream session."
    },
    "segment_id": {
     "type": "string",
     "pattern": "^[0-9a-f]{64}$",
     "description": "Deterministic SHA-256 segment identifier derived from stable segment identity
     ↪  fields. MUST remain stable across replay and backfill."
    },
    "segment_window_start_utc": {
     "type": "string",
     "format": "date-time",
     "description": "RFC 3339 UTC timestamp of the 30-second segment start boundary."
    },
    "segment_window_end_utc": {
     "type": "string",
     "format": "date-time",
     "description": "RFC 3339 UTC timestamp of the 30-second segment end boundary."
    },
    "timestamp_utc": {
     "type": "string",
     "format": "date-time",
     "description": "RFC 3339 UTC timestamp of inference event completion."
    },
    "media_source": {
     "type": "object",
     "required": [
      "stream_url",
      "codec",
      "resolution"
     ],
     "properties": {
      "stream_url": {
       "type": "string",
       "format": "uri"
      },
      "codec": {
       "type": "string",
       "enum": [
        "h264",
        "h265",
        "raw"
       ]
      },
      "resolution": {
       "type": "array",
       "items": {
        "type": "integer",
        "minimum": 1
```

```
      },
      "minItems": 2,
      "maxItems": 2,
      "description": "[width, height] in pixels."
    }
  },
  "additionalProperties": false
},
"segments": {
  "type": "array",
  "description": "Array of analyzed video segments."
},
"_active_arm": {
  "type": "string",
  "description": "Thompson Sampling arm identifier selected for this segment."
},
"_experiment_id": {
  "type": "integer",
  "description": "Persistent Store experiment row ID for this session."
},
"_expected_greeting": {
  "type": "string",
  "description": "Greeting rule text assigned to the active arm for semantic evaluation."
},
"_stimulus_time": {
  "type": [
    "number",
    "null"
  ],
  "description": "Drift-corrected UTC epoch seconds of stimulus injection, or null if no
  ↪  stimulus occurred in this segment."
},
"_au12_series": {
  "type": "array",
  "items": {
    "type": "object",
    "required": [
      "timestamp_s",
      "intensity"
    ],
    "properties": {
      "timestamp_s": {
        "type": "number"
      },
      "intensity": {
        "type": "number",
        "minimum": 0.0,
        "maximum": 1.0
      }
    },
    "additionalProperties": false
  },
  "description": "Per-frame AU12 bounded intensity observations from the Orchestrator
  ↪  Container."
},
"_x_max": {
  "type": [
    "number",
    "null"
  ],
```

```
    "description": "Per-subject maximum raw D_mouth/IOD ratio observed during calibration.
    ↪  Carried as calibration telemetry; \u00a77B reward computation consumes bounded AU12
    ↪  intensities and does not divide P90 by this raw ratio."
   },
   "_bandit_decision_snapshot": {
    "$ref": "#/$defs/BanditDecisionSnapshot",
    "description": "Pre-update Thompson Sampling selection snapshot captured in Module C at
    ↪  arm-selection time."
   },
   "_physiological_context": {
    "$ref": "#/$defs/PhysiologicalContext",
    "description": "Optional per-subject physiological context snapshot attached by Module C when
    ↪  eligible physiological data exists."
   }
  },
  "$defs": {
   "ArmPosterior": {
    "type": "object",
    "required": [
     "alpha",
     "beta"
    ],
    "properties": {
     "alpha": {
      "type": "number",
      "exclusiveMinimum": 0.0
     },
     "beta": {
      "type": "number",
      "exclusiveMinimum": 0.0
     }
    },
    "additionalProperties": false
   },
   "BanditDecisionSnapshot": {
    "type": "object",
    "required": [
     "selection_method",
     "selection_time_utc",
     "experiment_id",
     "policy_version",
     "selected_arm_id",
     "candidate_arm_ids",
     "posterior_by_arm",
     "expected_greeting"
    ],
    "properties": {
     "selection_method": {
      "type": "string",
      "enum": [
       "thompson_sampling"
      ]
     },
     "selection_time_utc": {
      "type": "string",
      "format": "date-time"
     },
     "experiment_id": {
      "type": "integer"
     },
```

```
      "policy_version": {
       "type": "string"
      },
      "selected_arm_id": {
       "type": "string"
      },
      "candidate_arm_ids": {
       "type": "array",
       "items": {
        "type": "string"
       },
       "minItems": 1,
       "uniqueItems": true
      },
      "posterior_by_arm": {
       "type": "object",
       "minProperties": 1,
       "additionalProperties": {
        "$ref": "#/$defs/ArmPosterior"
       }
      },
      "sampled_theta_by_arm": {
       "type": "object",
       "additionalProperties": {
        "type": "number",
        "minimum": 0.0,
        "maximum": 1.0
       }
      },
      "expected_greeting": {
       "type": "string"
      },
      "decision_context_hash": {
       "type": "string",
       "pattern": "^[0-9a-f]{64}$"
      },
      "random_seed": {
       "type": [
        "integer",
        "null"
       ]
      }
     },
     "additionalProperties": false
    },
    "PhysiologicalSnapshot": {
     "type": "object",
     "required": [
      "source_timestamp_utc",
      "freshness_s",
      "is_stale",
      "provider",
      "source_kind",
      "derivation_method",
      "window_s",
      "validity_ratio",
      "is_valid"
     ],
     "properties": {
      "rmssd_ms": {
       "type": [
```

```
        "number",
        "null"
      ],
      "minimum": 0.0
    },
    "heart_rate_bpm": {
      "type": [
        "integer",
        "null"
      ],
      "minimum": 20,
      "maximum": 300
    },
    "source_timestamp_utc": {
      "type": "string",
      "format": "date-time"
    },
    "freshness_s": {
      "type": "number",
      "minimum": 0.0
    },
    "is_stale": {
      "type": "boolean"
    },
    "provider": {
      "type": "string"
    },
    "source_kind": {
      "type": "string",
      "enum": [
        "ibi",
        "session"
      ]
    },
    "derivation_method": {
      "type": "string",
      "enum": [
        "provider",
        "server"
      ]
    },
    "window_s": {
      "type": "integer",
      "exclusiveMinimum": 0
    },
    "validity_ratio": {
      "type": "number",
      "minimum": 0.0,
      "maximum": 1.0
    },
    "is_valid": {
      "type": "boolean"
    }
  },
  "additionalProperties": false
},
"PhysiologicalContext": {
  "type": "object",
  "properties": {
    "streamer": {
      "oneOf": [
```

```
        {
         "type": "null"
        },
        {
         "$ref": "#/$defs/PhysiologicalSnapshot"
        }
       ]
      },
      "operator": {
       "oneOf": [
        {
         "type": "null"
        },
        {
         "$ref": "#/$defs/PhysiologicalSnapshot"
        }
       ]
      }
     },
     "additionalProperties": false,
     "anyOf": [
      {
       "required": [
        "streamer"
       ],
       "properties": {
        "streamer": {
         "$ref": "#/$defs/PhysiologicalSnapshot"
        }
       }
      },
      {
       "required": [
        "operator"
       ],
       "properties": {
        "operator": {
         "$ref": "#/$defs/PhysiologicalSnapshot"
        }
       }
      }
     ],
     "description": "Optional per-segment physiological context. When present, at least one of
     ↪  streamer or operator MUST contain a real PhysiologicalSnapshot; empty objects and
     ↪  null-only placeholders are invalid."
    }
   },
   "additionalProperties": false
  }
```

6.2 PhysiologicalSampleEvent

PhysiologicalSampleEvent JSON

```
{
 "$schema": "http://json-schema.org/draft-07/schema#",
 "title": "PhysiologicalChunkEvent",
 "type": "object",
 "required": [
  "unique_id",
  "event_type",
  "provider",
  "subject_role",
  "source_kind",
  "window_start_utc",
  "window_end_utc",
  "ingest_timestamp_utc",
  "payload"
 ],
 "properties": {
  "unique_id": {
   "type": "string",
   "format": "uuid"
  },
  "event_type": {
   "type": "string",
   "enum": [
    "physiological_chunk"
   ]
  },
  "provider": {
   "type": "string",
   "enum": [
    "oura"
   ]
  },
  "subject_role": {
   "type": "string",
   "enum": [
    "streamer",
    "operator"
   ]
  },
  "source_kind": {
   "type": "string",
   "enum": [
    "ibi",
    "session"
   ]
  },
  "window_start_utc": {
   "type": "string",
   "format": "date-time"
  },
  "window_end_utc": {
   "type": "string",
   "format": "date-time"
  },
  "ingest_timestamp_utc": {
   "type": "string",
```

```
      "format": "date-time"
     },
     "payload": {
      "type": "object",
      "required": [
       "sample_interval_s",
       "valid_sample_count",
       "expected_sample_count",
       "derivation_method"
      ],
      "properties": {
       "ibi_ms_items": {
        "type": "array",
        "items": {
         "type": "number",
         "exclusiveMinimum": 0.0
        }
       },
       "rmssd_items_ms": {
        "type": "array",
        "items": {
         "type": "number",
         "minimum": 0.0
        }
       },
       "heart_rate_items_bpm": {
        "type": "array",
        "items": {
         "type": "integer",
         "minimum": 20,
         "maximum": 300
        }
       },
       "motion_items": {
        "type": "array",
        "items": {
         "type": "number",
         "minimum": 0.0
        }
       },
       "sample_interval_s": {
        "type": "integer",
        "exclusiveMinimum": 0
       },
       "valid_sample_count": {
        "type": "integer",
        "minimum": 0
       },
       "expected_sample_count": {
        "type": "integer",
        "minimum": 0
       },
       "derivation_method": {
        "type": "string",
        "enum": [
         "provider",
         "server"
        ]
       }
      },
      "additionalProperties": false,
```

```
    "anyOf": [
     {
      "required": [
       "ibi_ms_items"
      ]
     },
     {
      "required": [
       "rmssd_items_ms"
      ]
     },
     {
      "required": [
       "heart_rate_items_bpm"
      ]
     }
    ]
   }
  },
  "additionalProperties": false
}
```

6.3 PhysiologicalContext

PhysiologicalContext JSON

```
{
 "$schema": "http://json-schema.org/draft-07/schema#",
 "title": "PhysiologicalContext",
 "type": "object",
 "properties": {
  "streamer": {
   "oneOf": [
    {
     "type": "null"
    },
    {
     "$ref": "#/$defs/PhysiologicalSnapshot"
    }
   ]
  },
  "operator": {
   "oneOf": [
    {
     "type": "null"
    },
    {
     "$ref": "#/$defs/PhysiologicalSnapshot"
    }
   ]
  }
 },
 "$defs": {
  "PhysiologicalSnapshot": {
   "type": "object",
   "required": [
```

```
      "source_timestamp_utc",
      "freshness_s",
      "is_stale",
      "provider",
      "source_kind",
      "derivation_method",
      "window_s",
      "validity_ratio",
      "is_valid"
    ],
    "properties": {
      "rmssd_ms": {
        "type": [
          "number",
          "null"
        ],
        "minimum": 0.0
      },
      "heart_rate_bpm": {
        "type": [
          "integer",
          "null"
        ],
        "minimum": 20,
        "maximum": 300
      },
      "source_timestamp_utc": {
        "type": "string",
        "format": "date-time"
      },
      "freshness_s": {
        "type": "number",
        "minimum": 0.0
      },
      "is_stale": {
        "type": "boolean"
      },
      "provider": {
        "type": "string"
      },
      "source_kind": {
        "type": "string",
        "enum": [
          "ibi",
          "session"
        ]
      },
      "derivation_method": {
        "type": "string",
        "enum": [
          "provider",
          "server"
        ]
      },
      "window_s": {
        "type": "integer",
        "exclusiveMinimum": 0
      },
      "validity_ratio": {
        "type": "number",
        "minimum": 0.0,
```

```
      "maximum": 1.0
     },
     "is_valid": {
      "type": "boolean"
     }
    },
    "additionalProperties": false
   }
  },
  "additionalProperties": false,
  "anyOf": [
   {
    "required": [
     "streamer"
    ],
    "properties": {
     "streamer": {
      "$ref": "#/$defs/PhysiologicalSnapshot"
     }
    }
   },
   {
    "required": [
     "operator"
    ],
    "properties": {
     "operator": {
      "$ref": "#/$defs/PhysiologicalSnapshot"
     }
    }
   }
  ],
  "description": "Per-segment physiological context. At least one role MUST contain a real
  ↪  PhysiologicalSnapshot; empty objects and null-only placeholders are invalid."
 }
```

6.4 BanditDecisionSnapshot

BanditDecisionSnapshot JSON

```
{
 "$schema": "http://json-schema.org/draft-07/schema#",
 "title": "BanditDecisionSnapshot",
 "type": "object",
 "required": [
  "selection_method",
  "selection_time_utc",
  "experiment_id",
  "policy_version",
  "selected_arm_id",
  "candidate_arm_ids",
  "posterior_by_arm",
  "expected_greeting"
 ],
 "properties": {
  "selection_method": {
```

```
      "type": "string",
      "enum": [
        "thompson_sampling"
      ]
    },
    "selection_time_utc": {
      "type": "string",
      "format": "date-time"
    },
    "experiment_id": {
      "type": "integer"
    },
    "policy_version": {
      "type": "string"
    },
    "selected_arm_id": {
      "type": "string"
    },
    "candidate_arm_ids": {
      "type": "array",
      "items": {
        "type": "string"
      },
      "minItems": 1,
      "uniqueItems": true
    },
    "posterior_by_arm": {
      "type": "object",
      "minProperties": 1,
      "additionalProperties": {
        "$ref": "#/$defs/ArmPosterior"
      }
    },
    "sampled_theta_by_arm": {
      "type": "object",
      "additionalProperties": {
        "type": "number",
        "minimum": 0.0,
        "maximum": 1.0
      }
    },
    "expected_greeting": {
      "type": "string"
    },
    "decision_context_hash": {
      "type": "string",
      "pattern": "^[0-9a-f]{64}$"
    },
    "random_seed": {
      "type": [
        "integer",
        "null"
      ]
    }
  },
  "$defs": {
    "ArmPosterior": {
      "type": "object",
      "required": [
        "alpha",
        "beta"
```

```
    ],
    "properties": {
     "alpha": {
      "type": "number",
      "exclusiveMinimum": 0.0
     },
     "beta": {
      "type": "number",
      "exclusiveMinimum": 0.0
     }
    },
    "additionalProperties": false
   }
  },
  "additionalProperties": false
 }
```

6.5 AttributionEvent

AttributionEvent JSON

```
{
 "$schema": "http://json-schema.org/draft-07/schema#",
 "title": "AttributionEvent",
 "type": "object",
 "required": [
  "event_id",
  "session_id",
  "segment_id",
  "event_type",
  "event_time_utc",
  "selected_arm_id",
  "expected_rule_text_hash",
  "semantic_method",
  "semantic_method_version",
  "reward_path_version",
  "bandit_decision_snapshot",
  "finality",
  "schema_version",
  "created_at"
 ],
 "properties": {
  "event_id": {
   "type": "string",
   "format": "uuid",
   "description": "Deterministic UUIDv5 derived from session_id, segment_id, event_type, and
   ↪  reward_path_version. Replay-stable and used for attribution backfill idempotency."
  },
  "session_id": {
   "type": "string",
   "pattern": "^[0-9a-f]{8}-[0-9a-f]{4}-4[0-9a-f]{3}-[89ab][0-9a-f]{3}-[0-9a-f]{12}$"
  },
  "segment_id": {
   "type": "string",
   "pattern": "^[0-9a-f]{64}$"
  },
```

```
    "event_type": {
     "type": "string",
     "enum": [
      "greeting_interaction"
     ]
    },
    "event_time_utc": {
     "type": "string",
     "format": "date-time"
    },
    "stimulus_time_utc": {
     "type": [
      "string",
      "null"
     ],
     "format": "date-time"
    },
    "selected_arm_id": {
     "type": "string"
    },
    "expected_rule_text_hash": {
     "type": "string",
     "pattern": "^[0-9a-f]{64}$"
    },
    "semantic_method": {
     "type": "string",
     "enum": [
      "cross_encoder",
      "llm_gray_band",
      "azure_llm_legacy"
     ]
    },
    "semantic_method_version": {
     "type": "string"
    },
    "semantic_p_match": {
     "type": [
      "number",
      "null"
     ],
     "minimum": 0.0,
     "maximum": 1.0
    },
    "semantic_reason_code": {
     "type": [
      "string",
      "null"
     ]
    },
    "reward_path_version": {
     "type": "string"
    },
    "bandit_decision_snapshot": {
     "$ref": "#/$defs/BanditDecisionSnapshot"
    },
    "evidence_flags": {
     "type": "array",
     "items": {
      "type": "string"
     },
     "uniqueItems": true
```

```
    },
    "finality": {
     "type": "string",
     "enum": [
      "online_provisional",
      "offline_final"
     ]
    },
    "schema_version": {
     "type": "string"
    },
    "created_at": {
     "type": "string",
     "format": "date-time"
    }
   },
   "$defs": {
    "ArmPosterior": {
     "type": "object",
     "required": [
      "alpha",
      "beta"
     ],
     "properties": {
      "alpha": {
       "type": "number",
       "exclusiveMinimum": 0.0
      },
      "beta": {
       "type": "number",
       "exclusiveMinimum": 0.0
      }
     },
     "additionalProperties": false
    },
    "BanditDecisionSnapshot": {
     "type": "object",
     "required": [
      "selection_method",
      "selection_time_utc",
      "experiment_id",
      "policy_version",
      "selected_arm_id",
      "candidate_arm_ids",
      "posterior_by_arm",
      "expected_greeting"
     ],
     "properties": {
      "selection_method": {
       "type": "string",
       "enum": [
        "thompson_sampling"
       ]
      },
      "selection_time_utc": {
       "type": "string",
       "format": "date-time"
      },
      "experiment_id": {
       "type": "integer"
      },
```

```
      "policy_version": {
       "type": "string"
      },
      "selected_arm_id": {
       "type": "string"
      },
      "candidate_arm_ids": {
       "type": "array",
       "items": {
        "type": "string"
       },
       "minItems": 1,
       "uniqueItems": true
      },
      "posterior_by_arm": {
       "type": "object",
       "minProperties": 1,
       "additionalProperties": {
        "$ref": "#/$defs/ArmPosterior"
       }
      },
      "sampled_theta_by_arm": {
       "type": "object",
       "additionalProperties": {
        "type": "number",
        "minimum": 0.0,
        "maximum": 1.0
       }
      },
      "expected_greeting": {
       "type": "string"
      },
      "decision_context_hash": {
       "type": "string",
       "pattern": "^[0-9a-f]{64}$"
      },
      "random_seed": {
       "type": [
        "integer",
        "null"
       ]
      }
     },
     "additionalProperties": false
    }
   },
   "additionalProperties": false
  }
```

6.6 OutcomeEvent

OutcomeEvent JSON

```
{
 "$schema": "http://json-schema.org/draft-07/schema#",
 "title": "OutcomeEvent",
 "type": "object",
 "required": [
  "outcome_id",
  "session_id",
  "outcome_type",
  "outcome_value",
  "outcome_time_utc",
  "source_system",
  "confidence",
  "finality",
  "schema_version",
  "created_at"
 ],
 "properties": {
  "outcome_id": {
   "type": "string",
   "format": "uuid",
   "description": "Deterministic UUIDv5 derived from session_id, outcome_type, outcome_time_utc,
   ↪  source_system, and source_event_ref. Replay-stable and used for attribution backfill
   ↪  idempotency."
  },
  "session_id": {
   "type": "string",
   "pattern": "^[0-9a-f]{8}-[0-9a-f]{4}-4[0-9a-f]{3}-[89ab][0-9a-f]{3}-[0-9a-f]{12}$"
  },
  "outcome_type": {
   "type": "string",
   "enum": [
    "creator_follow"
   ]
  },
  "outcome_value": {
   "type": "number"
  },
  "outcome_time_utc": {
   "type": "string",
   "format": "date-time"
  },
  "source_system": {
   "type": "string"
  },
  "source_event_ref": {
   "type": [
    "string",
    "null"
   ]
  },
  "confidence": {
   "type": "number",
   "minimum": 0.0,
   "maximum": 1.0
  },
  "finality": {
```

```
    "type": "string",
    "enum": [
     "online_provisional",
     "offline_final"
    ]
   },
   "schema_version": {
    "type": "string"
   },
   "created_at": {
    "type": "string",
    "format": "date-time"
   }
  },
  "additionalProperties": false
 }
```

6.7 EventOutcomeLink

EventOutcomeLink JSON

```
{
 "$schema": "http://json-schema.org/draft-07/schema#",
 "title": "EventOutcomeLink",
 "type": "object",
 "required": [
  "link_id",
  "event_id",
  "outcome_id",
  "lag_s",
  "horizon_s",
  "link_rule_version",
  "eligibility_flags",
  "finality",
  "schema_version",
  "created_at"
 ],
 "properties": {
  "link_id": {
   "type": "string",
   "format": "uuid",
   "description": "Deterministic UUIDv5 derived from event_id, outcome_id, and link_rule_version.
   ↪  Replay-stable and used for attribution backfill idempotency."
  },
  "event_id": {
   "type": "string",
   "format": "uuid"
  },
  "outcome_id": {
   "type": "string",
   "format": "uuid"
  },
  "lag_s": {
   "type": "number",
   "minimum": 0.0
  },
```

```
    "horizon_s": {
     "type": "number",
     "exclusiveMinimum": 0.0
    },
    "link_rule_version": {
     "type": "string"
    },
    "eligibility_flags": {
     "type": "array",
     "items": {
      "type": "string"
     },
     "uniqueItems": true
    },
    "finality": {
     "type": "string",
     "enum": [
      "online_provisional",
      "offline_final"
     ]
    },
    "schema_version": {
     "type": "string"
    },
    "created_at": {
     "type": "string",
     "format": "date-time"
    }
   },
   "additionalProperties": false
  }
```

6.8 AttributionScore

AttributionScore JSON

```
{
 "$schema": "http://json-schema.org/draft-07/schema#",
 "title": "AttributionScore",
 "type": "object",
 "required": [
  "score_id",
  "event_id",
  "attribution_method",
  "method_version",
  "score_raw",
  "finality",
  "schema_version",
  "created_at"
 ],
 "properties": {
  "score_id": {
   "type": "string",
   "format": "uuid",
```

```
    "description": "Deterministic UUIDv5 derived from event_id, outcome_id, attribution_method,
    ↪  method_version, and finality. Replay-stable and used for attribution backfill
    ↪  idempotency."
   },
   "event_id": {
    "type": "string",
    "format": "uuid"
   },
   "outcome_id": {
    "type": [
     "string",
     "null"
    ],
    "format": "uuid"
   },
   "attribution_method": {
    "type": "string"
   },
   "method_version": {
    "type": "string"
   },
   "score_raw": {
    "type": [
     "number",
     "null"
    ]
   },
   "score_normalized": {
    "type": [
     "number",
     "null"
    ]
   },
   "confidence": {
    "type": [
     "number",
     "null"
    ],
    "minimum": 0.0,
    "maximum": 1.0
   },
   "evidence_flags": {
    "type": "array",
    "items": {
     "type": "string"
    },
    "uniqueItems": true
   },
   "finality": {
    "type": "string",
    "enum": [
     "online_provisional",
     "offline_final"
    ]
   },
   "schema_version": {
    "type": "string"
   },
   "created_at": {
    "type": "string",
    "format": "date-time"
```

```
    }
  },
  "additionalProperties": false
}
```

SECTION 7

Mathematical specifications

7A AU12 facial action unit geometry

AU12 measures lip-corner pull driven by the zygomaticus major. In LSIE-MLF it is the primary facial-affect signal used for reward estimation.

Module C computes AU12 intensity from the 478 three-dimensional MediaPipe Face Mesh landmarks. Geometric normalization uses interocular distance (IOD) as the scale reference so the metric remains stable across head distance and pose.

7A.1 VARIABLE DICTIONARY

SYMBOL	CODE NAME	DEFINITION
L_i	`landmarks[i]`	3D vector (x_i, y_i, z_i) of MediaPipe landmark i.
L_{61}	`landmarks[61]`	Left lip corner landmark.
L_{291}	`landmarks[291]`	Right lip corner landmark.
L_{33}	`landmarks[33]`	Left eye outer corner.
L_{133}	`landmarks[133]`	Left eye inner corner.
L_{362}	`landmarks[362]`	Right eye inner corner.
L_{263}	`landmarks[263]`	Right eye outer corner.
E_L	`e_l`	Geometric center of left eye midpoint(L_33,L_133).
E_R	`e_r`	Geometric center of right eye midpoint(L_362,L_263).
IOD	`iod`	3D Euclidean distance between E_L and E_R.
D_{mouth}	`d_mouth`	3D Euclidean distance between lip corners.
B_{neutral}	`self.b_neutral`	Neutral baseline average of D_mouth/IOD.
$I_{\text{AU12}}(t)$	`i_au12`	Normalized AU12 intensity at time t.
α	`self.alpha`	Empirical linear projection multiplier. Default 6.0; calibrated so a Duchenne deviation of approximately 0.15 maps to approximately 0.90 before tanh bounding.
$Score_{\text{AU12}}(t)$	`score_au12`	Final AU12 FACS score in range 0–5.
$Score_{\text{AU12,bounded}}(t)$	`compute_bounded_intensity()`	AU12 intensity bounded to [0.0, 1.0] via tanh soft-saturation. Output method for the fractional Beta-Bernoulli Thompson Sampling reward pipeline.

STEP 1: TOPOLOGICAL LANDMARK EXTRACTION

The anatomical points required for AU12 computation are extracted from the MediaPipe landmark array.

- Mouth corners: L_61 (left) and L_291 (right)
- Left eye bounds: L_33 outer and L_133 inner
- Right eye bounds: L_362 inner and L_263 outer

STEP 2: DERIVATION OF INTEROCULAR DISTANCE (IOD)

Eye centers are computed as midpoints between inner and outer eye landmarks and used to calculate the normalization scalar.

$$E_L = ((x_{33}+x_{133})/2, (y_{33}+y_{133})/2, (z_{33}+z_{133})/2); \quad E_R = ((x_{362}+x_{263})/2, (y_{362}+y_{263})/2, (z_{362}+z_{263})/2)$$

$$IOD = \sqrt{(E_R.x - E_L.x)^2 + (E_R.y - E_L.y)^2 + (E_R.z - E_L.z)^2}$$

STEP 3: AU12 DISTANCE AND BASELINE CALIBRATION

The raw AU12 distance equals the Euclidean distance between lip corners.

A neutral baseline B_neutral is computed from the rolling average of D_mouth/IOD during the calibration phase.

The bounded output maps [0, +infinity) to [0, 1) via tanh soft-saturation, providing a continuous gradient at the boundaries suitable for fractional Beta-Bernoulli posterior updates. With alpha_scale=6.0, a Duchenne smile (deviation approximately 0.15) maps to tanh(0.9) approximately 0.72, and laughter (deviation approximately 0.20) maps to tanh(1.2) approximately 0.83.

$$D_{\text{mouth}} = \sqrt{(x_{291} - x_{61})^2 + (y_{291} - y_{61})^2 + (z_{291} - z_{61})^2}$$

$$I_{\text{AU12}}(t) = (D_{\text{mouth}}(t)/IOD(t)) - B_{\text{neutral}}$$

$$Score_{\text{AU12}}(t) = \min\left(5, \max(0, \alpha \cdot I_{\text{AU12}}(t))\right)$$

$$Score_{\text{AU12,bounded}}(t) = \tanh\big(\alpha \cdot \max(0,\ I_{\text{AU12}})\big)$$

7A.2 AU12NORMALIZER — REFERENCE IMPLEMENTATION

Reference implementation **PYTHON**

```python
from __future__ import annotations
import math
from typing import Any
import numpy as np
import numpy.typing as npt

EPSILON: float = 1e-6
```

```python
DEFAULT_ALPHA_SCALE: float = 6.0  # §7A: 0.90 / 0.15 = 6.0


class AU12Normalizer:
    def __init__(self, alpha: float = DEFAULT_ALPHA_SCALE) -> None:
        self.alpha: float = alpha
        self.b_neutral: float | None = None
        self.calibration_buffer: list[float] = []

    def _extract_geometry(
        self, landmarks: npt.NDArray[np.floating[Any]]
    ) -> tuple[float, float] | None:
        left_eye_outer = landmarks[33]
        left_eye_inner = landmarks[133]
        right_eye_inner = landmarks[362]
        right_eye_outer = landmarks[263]
        left_lip = landmarks[61]
        right_lip = landmarks[291]

        e_l = (left_eye_outer + left_eye_inner) / 2.0
        e_r = (right_eye_inner + right_eye_outer) / 2.0
        iod: float = float(np.linalg.norm(e_r - e_l))
        if iod < EPSILON:
            return None
        d_mouth: float = float(np.linalg.norm(right_lip - left_lip))
        return (iod, d_mouth)

    def _update_calibration(self, ratio: float) -> None:
        self.calibration_buffer.append(ratio)
        self.b_neutral = float(np.mean(self.calibration_buffer))

    def compute_raw_ratio(
        self, landmarks: npt.NDArray[np.floating[Any]]
    ) -> float | None:
        geom = self._extract_geometry(landmarks)
        if geom is None:
            return None
        iod, d_mouth = geom
        return d_mouth / iod

    def compute_intensity(
        self, landmarks: npt.NDArray[np.floating[Any]], is_calibrating: bool = False
    ) -> float:
        geom = self._extract_geometry(landmarks)
        if geom is None:
            return 0.0
        iod, d_mouth = geom
        ratio = d_mouth / iod
        if is_calibrating:
            self._update_calibration(ratio)
            return 0.0
        if self.b_neutral is None:
            raise ValueError("Baseline not calibrated")
        score = self.alpha * (ratio - self.b_neutral)
        return float(min(max(score, 0.0), 5.0))

    def compute_bounded_intensity(
        self, landmarks: npt.NDArray[np.floating[Any]], is_calibrating: bool = False
    ) -> float:
        geom = self._extract_geometry(landmarks)
        if geom is None:
```

```
            return 0.0
        iod, d_mouth = geom
        ratio = d_mouth / iod
        if is_calibrating:
            self._update_calibration(ratio)
            return 0.0
        if self.b_neutral is None:
            raise ValueError("Baseline not calibrated")
        deviation = max(0.0, ratio - self.b_neutral)
        return float(math.tanh(self.alpha * deviation))
```

7B Thompson sampling reward pipeline

LSIE-MLF uses Thompson Sampling to select greeting strategies that maximize creator engagement. The reward is derived from AU12 smile intensity (§7A), which serves as a dense proxy for the sparse creator-follow event.

The reward pipeline applies a binary semantic gate so only encounters where the greeting was actually delivered contribute to posterior updates. Non-attributable smiles do not update the posterior.

§7B is the sole normative reward path for v3.4. Physiological co-modulation (§7C), observational acoustic analytics (§7D), semantic probability scores beyond the binary gate (§8), and attribution analytics (§7E) are observational only and must not modify `compute_reward()`, `RewardResult`, Thompson Sampling posterior updates, or arm-selection logic.

7B.1 VARIABLE DICTIONARY

SYMBOL	CODE NAME	DEFINITION
t_{stim}	`stimulus_time_s`	Drift-corrected UTC epoch timestamp of greeting stimulus injection.
W_{measure}	`extract_stimulus_window()`	Stimulus-locked measurement window [*t_stim* + 0.5s, *t_stim* + 5.0s].
W_{baseline}	`extract_stimulus_-window(baseline)`	Pre-stimulus baseline window [*t_stim* - 5.0s, *t_stim* - 2.0s].
P_{90}	`p90_intensity`	90th percentile of bounded AU12 intensities within *W_measure*.
G_t	`semantic_gate`	Binary semantic validity gate: 1 if `is_match` is True, 0 otherwise.
r_t	`gated_reward`	Gated reward signal: *r_t* = *P_90* × *G_t*. Range [0.0, 1.0].
α	`posterior_alpha`	Beta distribution alpha parameter (success count). Prior: 1.0.
β	`posterior_beta`	Beta distribution beta parameter (failure count). Prior: 1.0.
$B_{\text{neutral,pre}}$	`au12_baseline_pre`	Mean bounded AU12 intensity in *W_baseline*, or null when no pre-stimulus frames exist.

STEP 1: SIGNAL SOURCE AND BOUNDED OUTPUT

The reward pipeline consumes per-frame bounded AU12 intensities from `compute_bounded_intensity()` (§7A). Each observation is a `{timestamp_s, intensity}` pair with intensity in [0.0, 1.0].

STEP 2: STIMULUS WINDOW EXTRACTION

The measurement window is defined as [*t_stim* + 0.5s, *t_stim* + 5.0s]. The 0.5-second onset delay accounts for the physiological latency between hearing the greeting and producing a facial response. The 5.0-second end bound captures the full smile trajectory including onset, apex, and initial decay.

A pre-stimulus baseline window [*t_stim* - 5.0s, *t_stim* - 2.0s] is computed for diagnostic purposes. The 2.0-second guard prevents contamination from anticipatory expressions triggered by the operator's preparation to speak.

$$W_{\text{measure}} = \{(t_i, s_i) \in \text{au12_series} \mid t_{\text{stim}} + 0.5 \leq t_i \leq t_{\text{stim}} + 5.0\}$$

$$W_{\text{baseline}} = \{(t_i, s_i) \in \text{au12_series} \mid t_{\text{stim}} - 5.0 \leq t_i \leq t_{\text{stim}} - 2.0\}$$

STEP 3: P90 AGGREGATION AND GATED REWARD

The 90th percentile is used instead of the mean or maximum to reduce sensitivity to single-frame noise spikes while still capturing the peak smile response. If the measurement window contains zero frames, P90 defaults to 0.0.

The semantic gate *G_t* is binary and comes from the `is_match` output (§8). If the greeting was not delivered, the reward is zero regardless of smile intensity.

$$P_{90} = \text{percentile}_{90}\big(\{s_i \mid (t_i, s_i) \in W_{\text{measure}}\}\big)$$

$$G_t = \begin{cases} 1 & \text{if is_match} = \text{True} \\ 0 & \text{otherwise} \end{cases}$$

$$r_t = P_{90} \times G_t$$

STEP 4: FRACTIONAL BETA-BERNOULLI POSTERIOR UPDATE

Each arm maintains a Beta posterior initialized with a uniform prior Beta(1, 1). After each encounter, the posterior is updated with the fractional reward *r_t* ∈ [0, 1]. Unlike standard Bernoulli updates that require binary 0/1 outcomes, the fractional update preserves the continuous gradient of the bounded AU12 signal.

Arm selection uses Thompson Sampling: for each arm, a value is sampled from its current Beta posterior, and the arm with the highest sampled value is selected for the next encounter.

$$\alpha \leftarrow \alpha + r_t, \quad \beta \leftarrow \beta + (1 - r_t)$$

$$a^* = \arg\max_a \theta_a, \quad \theta_a \sim \text{Beta}(\alpha_a, \beta_a)$$

7B.2 REWARD PIPELINE AND THOMPSON SAMPLING — REFERENCE IMPLEMENTATION

Reference implementation **PYTHON**

```python
from __future__ import annotations
import random
from dataclasses import dataclass
from typing import Optional
import numpy as np

STIMULUS_WINDOW_START: float = 0.5   # seconds after stimulus
STIMULUS_WINDOW_END: float = 5.0     # seconds after stimulus
BASELINE_WINDOW_START: float = -5.0  # seconds before stimulus
BASELINE_WINDOW_END: float = -2.0    # seconds before stimulus
DEFAULT_PRIOR_ALPHA: float = 1.0
DEFAULT_PRIOR_BETA: float = 1.0


@dataclass
class RewardResult:
    gated_reward: float
    p90_intensity: float
    semantic_gate: int
    n_frames_in_window: int
    au12_baseline_pre: Optional[float]


def extract_stimulus_window(
    au12_series: list[dict], stimulus_time_s: float,
    start_offset: float = STIMULUS_WINDOW_START,
    end_offset: float = STIMULUS_WINDOW_END,
) -> list[dict]:
    t_start = stimulus_time_s + start_offset
    t_end = stimulus_time_s + end_offset
    return [o for o in au12_series if t_start ≤ o["timestamp_s"] ≤ t_end]


def compute_reward(
    au12_series: list[dict], stimulus_time_s: float,
    is_match: bool,
) -> RewardResult:
    window = extract_stimulus_window(au12_series, stimulus_time_s)
    n_frames = len(window)

    if n_frames == 0:
        p90 = 0.0
    else:
        p90 = float(np.percentile([o["intensity"] for o in window], 90))


    gate = 1 if is_match else 0
    gated = p90 * gate

    baseline_obs = extract_stimulus_window(
        au12_series, stimulus_time_s,
        start_offset=BASELINE_WINDOW_START, end_offset=BASELINE_WINDOW_END,
    )
    baseline = (
        float(np.mean([o["intensity"] for o in baseline_obs]))
        if baseline_obs else None
```

```
    )

    return RewardResult(
        gated_reward=gated, p90_intensity=p90, semantic_gate=gate,
        n_frames_in_window=n_frames, au12_baseline_pre=baseline,
    )


def thompson_sample_arm(arms: dict[str, tuple[float, float]]) -> str:
    samples = {
        arm_id: random.betavariate(alpha, beta)
        for arm_id, (alpha, beta) in arms.items()
    }
    return max(samples, key=samples.get)


def update_posterior(
    alpha: float, beta: float, reward: float
) -> tuple[float, float]:
    return alpha + reward, beta + (1.0 - reward)
```

7C Physiological co-modulation analytics

This section defines physiological co-modulation as a rolling Pearson correlation between streamer and operator RMSSD time series.

RMSSD snapshots are derived in Module C from normalized physiological chunks and persisted per segment. This analytics path is observational only under the §7B reward-path invariance rule.

7C.1 VARIABLE DICTIONARY

SYMBOL	CODE NAME	DEFINITION
$R_s^{(k)}$	`streamer_rmssd_bins[k]`	Streamer RMSSD in a 1-minute resampled analysis bin.
$R_o^{(k)}$	`operator_rmssd_bins[k]`	Operator RMSSD in a 1-minute resampled analysis bin.
C_{mod}	`co_modulation_index`	Pearson correlation between aligned streamer and operator RMSSD sequences.
N	`n_paired_observations`	Number of aligned paired observations in the analysis window.
$coverage$_ratio	`coverage_ratio`	Fraction of the analysis window with valid paired observations.
MIN_COMOD_PAIRS	`MIN_COMOD_PAIRS`	Minimum aligned pairs required for a valid correlation.
$COMOD$_WINDOW_MINUTES	`COMOD_WINDOW_MINUTES`	Rolling analysis window duration in minutes.

STEP 1: DATA SOURCE AND ELIGIBILITY

Use per-segment physiological snapshots from `physiology_log`.

A snapshot is eligible only when `rmssd_ms` is not null, `is_valid = TRUE`, and `is_stale = FALSE`.

STEP 2: WINDOWING, RESAMPLING, AND TIME ALIGNMENT

For each subject, select eligible snapshots in the 10-minute window ending at `window_end_utc`.

Resample each series to 1-minute mean bins and inner-join on timestamp.

$$R_s = \text{resample}_{1\text{min}}(\text{streamer_rmssd})$$

$$R_o = \text{resample}_{1\text{min}}(\text{operator_rmssd})$$

STEP 3: PEARSON CORRELATION AND NULL GATING

Compute Pearson correlation on the aligned RMSSD series. Positive values indicate alignment; negative values indicate divergence.

- If N < 4 (MIN_COMOD_PAIRS), co_modulation_index = null.
- If either aligned series has zero variance, co_modulation_index = null.
- If N >= 4 and both aligned series have non-zero variance, compute co_modulation_index using scipy.stats.pearsonr.

$$C_{\text{mod}} = \frac{\sum(R_s^{(k)} - \bar{R}_s)(R_o^{(k)} - \bar{R}_o)}{\sqrt{\sum(R_s^{(k)} - \bar{R}_s)^2 \sum(R_o^{(k)} - \bar{R}_o)^2}}$$

STEP 4: COVERAGE RATIO AND PERSISTENCE

Coverage ratio measures the fraction of the window with valid paired observations.

Persist the result even when `co_modulation_index` is null.

- co_modulation_index
- n_paired_observations
- coverage_ratio
- streamer_rmssd_mean
- operator_rmssd_mean
- window_start_utc
- window_end_utc

$$\text{coverage_ratio} = \frac{N}{\max(|R_s|, |R_o|, 1)}$$

STEP 5: SCOPE CONSTRAINT

This analytics path is observational only under the §7B reward-path invariance rule.

7C.2 PHYSIOLOGICAL CO-MODULATION — REFERENCE IMPLEMENTATION

Reference implementation PYTHON

```python
from __future__ import annotations

from dataclasses import dataclass
from typing import Optional

import pandas as pd
from scipy.stats import pearsonr

MIN_COMOD_PAIRS: int = 4
COMOD_WINDOW_MINUTES: int = 10
RESAMPLE_RULE: str = "1min"


@dataclass(frozen=True)
class CoModulationResult:
    co_modulation_index: Optional[float]
    n_paired_observations: int
    coverage_ratio: float
    streamer_rmssd_mean: Optional[float]
    operator_rmssd_mean: Optional[float]


def _resample_rmssd(window_df: pd.DataFrame, subject_role: str) -> pd.Series:
    role_df = (
        window_df.loc[
            window_df["subject_role"] == subject_role,
            ["source_timestamp_utc", "rmssd_ms"],
        ]
        .dropna(subset=["rmssd_ms"])
        .sort_values("source_timestamp_utc")
    )

    if role_df.empty:
        return pd.Series(dtype="float64")

    return (
        role_df.set_index("source_timestamp_utc")["rmssd_ms"]
        .resample(RESAMPLE_RULE)
        .mean()
        .dropna()
    )


def compute_comodulation(
    physiology_log: pd.DataFrame,
    window_end_utc: pd.Timestamp,
) -> CoModulationResult:
    window_end_utc = pd.Timestamp(window_end_utc, tz="UTC")
    window_start_utc = window_end_utc - pd.Timedelta(minutes=COMOD_WINDOW_MINUTES)

    eligible = physiology_log.copy()
    eligible["source_timestamp_utc"] = pd.to_datetime(
```

```
        eligible["source_timestamp_utc"], utc=True
    )
    eligible = eligible.loc[
        (eligible["source_timestamp_utc"] > window_start_utc)
        & (eligible["source_timestamp_utc"] ≤ window_end_utc)
        & eligible["rmssd_ms"].notna()
        & (eligible["is_valid"] == True)
        & (eligible["is_stale"] == False)
    ]

    streamer = _resample_rmssd(eligible, "streamer")
    operator = _resample_rmssd(eligible, "operator")

    aligned = pd.concat(
        [streamer.rename("streamer"), operator.rename("operator")],
        axis=1,
        join="inner",
    ).dropna()

    n_pairs = int(len(aligned))
    coverage_ratio = float(
        n_pairs / max(len(streamer), len(operator), 1)
    )

    streamer_mean = float(streamer.mean()) if not streamer.empty else None
    operator_mean = float(operator.mean()) if not operator.empty else None

    if n_pairs < MIN_COMOD_PAIRS:
        return CoModulationResult(
            co_modulation_index=None,
            n_paired_observations=n_pairs,
            coverage_ratio=coverage_ratio,
            streamer_rmssd_mean=streamer_mean,
            operator_rmssd_mean=operator_mean,
        )

    if aligned["streamer"].nunique() < 2 or aligned["operator"].nunique() < 2:
        return CoModulationResult(
            co_modulation_index=None,
            n_paired_observations=n_pairs,
            coverage_ratio=coverage_ratio,
            streamer_rmssd_mean=streamer_mean,
            operator_rmssd_mean=operator_mean,
        )

    co_modulation_index, _p_value = pearsonr(
        aligned["streamer"], aligned["operator"]
    )

    return CoModulationResult(
        co_modulation_index=float(co_modulation_index),
        n_paired_observations=n_pairs,
        coverage_ratio=coverage_ratio,
        streamer_rmssd_mean=streamer_mean,
        operator_rmssd_mean=operator_mean,
    )
```

7D Observational acoustic analytics

This section defines stimulus-windowed observational acoustic analytics from streamer speech audio.

The metrics are derived in Module D from the existing 16 kHz speech stream and persisted as derived analytical outputs. This analytics path is observational only under the §7B reward-path invariance rule.

7D.1 VARIABLE DICTIONARY

SYMBOL	CODE NAME	DEFINITION
τ_{voice}	`ACOUSTIC_VOICING_THRESHOLD`	Minimum Praat voicing threshold for a frame to be treated as voiced.
$T_{\text{voice,min}}$	`ACOUSTIC_MIN_VOICED_COVERAGE_S`	Minimum cumulative voiced coverage required for F0 validity inside a window.
$T_{\text{island,min}}$	`ACOUSTIC_MIN_STABLE_ISLAND_S`	Minimum duration of one continuous voiced island required for F0 validity inside a window.
$N_{\text{peaks,min}}$	`ACOUSTIC_MIN_PERIODIC_PEAKS`	Minimum periodic peaks required for perturbation validity inside a selected voiced island.
$V(W)$	`voiced_frames(window)`	Frames in the analysis window whose voicing strength meets or exceeds the voicing threshold.
$C_{\text{voice}}(W)$	`voiced_coverage_s(window)`	Sum of durations of voiced frames in the analysis window.
$L_{\text{voice}}(W)$	`longest_voiced_island_s(window)`	Duration of the longest contiguous voiced island in the analysis window.
$P(W)$	`perturbation_span(window)`	Selected contiguous voiced island used for jitter/shimmer computation; it must satisfy perturbation validity constraints.
Δ_{ST}	`f0_delta_semitones`	Semitone difference between measurement-window and baseline-window mean F0.

STEP 1: STIMULUS WINDOWS AND VOICED-ONLY MASKING

Use the same stimulus-locked windows already defined by §7B.

For each analysis window, mark frames as voiced only when voicing strength is greater than or equal to 0.45, compute cumulative voiced coverage, compute the longest contiguous voiced island, and exclude unvoiced frames from all voiced-only aggregates.

- ACOUSTIC_VOICING_THRESHOLD = 0.45
- ACOUSTIC_MIN_VOICED_COVERAGE_S = 1.0
- ACOUSTIC_MIN_STABLE_ISLAND_S = 0.2
- ACOUSTIC_MIN_PERIODIC_PEAKS = 4

$$W_{\text{measure}} = [t_{\text{stim}} + 0.5\text{s},\ t_{\text{stim}} + 5.0\text{s}]$$

$$W_{\text{baseline}} = [t_{\text{stim}} - 5.0\text{s},\ t_{\text{stim}} - 2.0\text{s}]$$

$$V(W) = \{\text{frame}_i \in W \mid \text{voicing_strength}(\text{frame}_i) \geq \tau_{\text{voice}}\}$$

$$C_{\text{voice}}(W) = \sum_{\text{frame}_i \in V(W)} \text{duration}(\text{frame}_i)$$

$$L_{\text{voice}}(W) = \text{max_contiguous_duration}(V(W))$$

STEP 2: VALIDITY GATING AND PERTURBATION-SPAN SELECTION

For each window W, F0 validity requires minimum cumulative voiced coverage and a minimum stable voiced island.

Perturbation validity requires F0 validity plus at least one contiguous voiced island containing the minimum periodic peaks and computable Praat local jitter/local shimmer values. If multiple eligible perturbation spans exist, use the longest eligible span.

- f0_valid(W) = true iff C_voice(W) >= ACOUSTIC_MIN_VOICED_COVERAGE_S and L_voice(W) >= ACOUSTIC_MIN_STABLE_ISLAND_S
- perturbation_valid(W) = true iff f0_valid(W) is true, at least one contiguous voiced island contains >= ACOUSTIC_MIN_PERIODIC_PEAKS periodic peaks, and Praat local jitter / local shimmer are computable on the selected perturbation span
- Otherwise the corresponding validity flag is false

STEP 3: SUMMARY STATISTICS AND DELTA FORMULAS

Voiced coverage outputs are the cumulative voiced coverage in the measure and baseline windows.

If F0 validity is true, compute arithmetic means only over voiced frames with F0 greater than zero. If perturbation validity is true, compute Praat local jitter and local shimmer on the selected perturbation span.

- If `f0_valid(W) = false`, the corresponding F0 mean is null.
- If `perturbation_valid(W) = false`, the corresponding perturbation means are null.
- If either F0 mean is null, `f0_delta_semitones` is null.
- If `f0_mean_baseline_hz` <= 0, `f0_delta_semitones` is null.
- If either jitter mean is null, `jitter_delta` is null.
- If either shimmer mean is null, `shimmer_delta` is null.
- Session-level z-scoring or population normalization must not be persisted at insertion time. The Persistent Store retains raw means, validity flags, coverage values, and canonical deltas only.

$$f0_mean_measure_hz = \text{mean}(F0(\text{frame}_i) \mid \text{frame}_i \in V(W_{\text{measure}}),\ F0(\text{frame}_i) > 0)$$

$$f0_mean_baseline_hz = \text{mean}(F0(\text{frame}_i) \mid \text{frame}_i \in V(W_{\text{baseline}}),\ F0(\text{frame}_i) > 0)$$

$$\Delta_{\text{ST}} = 12 \cdot \log_2 \left(\frac{f0_mean_measure_hz}{f0_mean_baseline_hz} \right)$$

$$\text{jitter_delta} = \text{jitter_mean_measure} - \text{jitter_mean_baseline}$$

$$\text{shimmer_delta} = \text{shimmer_mean_measure} - \text{shimmer_mean_baseline}$$

STEP 4: NULL STIMULUS BEHAVIOR

If `_stimulus_time` is null, Module D emits deterministic false/null observational acoustic outputs instead of attempting stimulus-window analysis.

- f0_valid_measure = false
- f0_valid_baseline = false
- perturbation_valid_measure = false
- perturbation_valid_baseline = false
- voiced_coverage_measure_s = 0.0
- voiced_coverage_baseline_s = 0.0
- all mean and delta fields = null

STEP 5: SCOPE CONSTRAINT

This analytics path is observational only under the §7B reward-path invariance rule.

7D.2 OBSERVATIONAL ACOUSTIC ANALYTICS — REFERENCE IMPLEMENTATION

Reference implementation PYTHON

```python
from __future__ import annotations

from dataclasses import dataclass
from math import log2
from typing import Optional

ACOUSTIC_VOICING_THRESHOLD: float = 0.45
ACOUSTIC_MIN_VOICED_COVERAGE_S: float = 1.0
ACOUSTIC_MIN_STABLE_ISLAND_S: float = 0.2
ACOUSTIC_MIN_PERIODIC_PEAKS: int = 4

STIMULUS_WINDOW_START: float = 0.5
STIMULUS_WINDOW_END: float = 5.0
BASELINE_WINDOW_START: float = -5.0
BASELINE_WINDOW_END: float = -2.0


@dataclass(frozen=True)
class AcousticAnalyticsResult:
    f0_valid_measure: bool
    f0_valid_baseline: bool
    perturbation_valid_measure: bool
    perturbation_valid_baseline: bool
    voiced_coverage_measure_s: float
    voiced_coverage_baseline_s: float
    f0_mean_measure_hz: Optional[float]
    f0_mean_baseline_hz: Optional[float]
    f0_delta_semitones: Optional[float]
```

```
    jitter_mean_measure: Optional[float]
    jitter_mean_baseline: Optional[float]
    jitter_delta: Optional[float]
    shimmer_mean_measure: Optional[float]
    shimmer_mean_baseline: Optional[float]
    shimmer_delta: Optional[float]


def compute_f0_delta_semitones(
    f0_mean_measure_hz: Optional[float],
    f0_mean_baseline_hz: Optional[float],
) -> Optional[float]:
    if f0_mean_measure_hz is None or f0_mean_baseline_hz is None:
        return None
    if f0_mean_baseline_hz ≤ 0:
        return None
    return 12.0 * log2(f0_mean_measure_hz / f0_mean_baseline_hz)


def null_acoustic_result() -> AcousticAnalyticsResult:
    return AcousticAnalyticsResult(
        f0_valid_measure=False,
        f0_valid_baseline=False,
        perturbation_valid_measure=False,
        perturbation_valid_baseline=False,
        voiced_coverage_measure_s=0.0,
        voiced_coverage_baseline_s=0.0,
        f0_mean_measure_hz=None,
        f0_mean_baseline_hz=None,
        f0_delta_semitones=None,
        jitter_mean_measure=None,
        jitter_mean_baseline=None,
        jitter_delta=None,
        shimmer_mean_measure=None,
        shimmer_mean_baseline=None,
        shimmer_delta=None,
    )
```

7E Event→Outcome attribution policy

This section defines cross-signal attribution as a derived analytical policy layer spanning semantic scoring, baseline-aware AU12 diagnostics, deterministic event→outcome linkage, and lag-aware observational synchrony analytics.

This analytics path is observational only under the §7B reward-path invariance rule.

7E.1 VARIABLE DICTIONARY

SYMBOL	CODE NAME	DEFINITION
$segment_{\text{id}}$	`segment_id`	Deterministic SHA-256 segment identifier derived from stable segment identity fields.
H_{attr}	`attribution_horizon_s`	Maximum eligible event→outcome attribution horizon in seconds.
lag_s	`lag_s`	Time difference between outcome timestamp and event timestamp in seconds.
p_{match}	`confidence_score`	Semantic probability estimate produced by the active deterministic semantic method.
r_t^{soft}	`soft_reward_candidate`	Observational soft semantic reward candidate persisted under §7E only.
B_{pre}	`au12_baseline_pre`	Mean bounded AU12 intensity in the pre-stimulus baseline window.
L_{p90}	`au12_lift_p90`	Baseline-aware lift using the post-stimulus P90 bounded AU12 intensity.
L_{peak}	`au12_lift_peak`	Baseline-aware lift using the post-stimulus peak bounded AU12 intensity.
$\Delta t_{\text{peak,ms}}$	`au12_peak_latency_ms`	Latency from stimulus timestamp to peak AU12 observation in milliseconds.
$\rho(\ell)$	`lagged_correlation`	Lagged Pearson correlation at lag \ell.
S	`sync_peak_corr`	Maximum lagged correlation over the configured lag search range.
$finality$	`finality`	Attribution record lifecycle state: online_provisional or offline_final.
r_t^{proxy}	`proxy_reward`	The observational proxy reward candidate used for attribution scoring, typically the §7B gated_reward or a documented diagnostic reward component.
x_t	`source_series`	First time-aligned observational signal used in lag-aware synchrony analytics.
y_t	`target_series`	Second time-aligned observational signal used in lag-aware synchrony analytics.

STEP 1: STABLE EVENT IDENTITY AND SCOPE

Each dispatched interaction segment must carry a deterministic segment identity so reprocessing and offline linkage remain idempotent.

A stable event key is derived in Module C before dispatch and reused by Module E when constructing AttributionEvent records.

$$\text{segment_id} = \text{SHA256}(\text{session_id} \parallel \text{segment_window_start_utc} \parallel \text{segment_window_end_utc})$$

STEP 2: OUTCOME DEFINITION AND ELIGIBILITY LINKING

Delayed outcomes such as creator_follow are persisted as OutcomeEvent records and linked to prior events by deterministic eligibility rules.

Eligibility linkage is many-to-many and defines candidate event→outcome pairs; it does not assert unique causal proof.

$$\text{lag}_s = t_{\text{outcome}} - t_{\text{event}}$$

$$0 \leq \text{lag}_s \leq H_{\text{attr}}$$

STEP 3: SEMANTIC ATTRIBUTION SCORES

The semantic scorer outputs a probability estimate *p_match* together with a bounded reason code and final boolean gate.

In v3.4 the soft semantic reward is persisted only as an observational candidate and does not modulate §7B reward updates.

$$r_t^{\text{soft}} = p_{\text{match}} \cdot r_t^{\text{proxy}}$$

$$P(\text{match} \mid \text{audio}, \text{rule}) = \sum_{k=1}^{K} P(\hat{u}_k \mid \text{audio})\, P_\phi(\text{match} \mid \hat{u}_k, \text{rule})$$

STEP 4: AU12 BASELINE-AWARE ATTRIBUTION METRICS

To distinguish stimulus-linked smiles from smiles already underway, attribution analytics persist baseline-aware AU12 lift metrics derived from the same windows already defined by §7B.

These metrics are diagnostic and do not replace the §7B gated reward in v3.4.

$$L_{\text{p90}} = \max(0, P_{90,post} - B_{\text{pre}})$$

$$L_{\text{peak}} = \max(0, P_{\text{max,post}} - B_{\text{pre}})$$

$$\Delta t_{\text{peak,ms}} = 1000 \cdot (t_{\text{peak}} - t_{\text{stim}})$$

STEP 5: LAG-AWARE SYNCHRONY ANALYTICS

Before more complex warping methods are introduced, attribution analytics persist a lagged-correlation baseline for observational synchrony.

These synchrony metrics remain observational and do not affect reward computation.

$$\rho(\ell) = \text{corr}(x_t, y_{t+\ell}), \qquad \ell \in [0, L]$$

$$S = \max_{\ell \in [0,L]} \rho(\ell)$$

STEP 6: FINALITY AND SCOPE CONSTRAINT

Attribution records transition from `online_provisional` to `offline_final` after attribution horizons close and finalization jobs complete.

All §7E outputs are analytical only under the §7B reward-path invariance rule.

- Allowed finality states: online_provisional, offline_final.
- Closed horizons do not reopen unless explicit reprocessing is invoked.

7E.2 ATTRIBUTION ANALYTICS — REFERENCE IMPLEMENTATION

Reference implementation PYTHON

```python
from __future__ import annotations

import hashlib
from dataclasses import dataclass
from typing import Optional

import numpy as np
from scipy.stats import pearsonr

STIMULUS_WINDOW_START: float = 0.5
STIMULUS_WINDOW_END: float = 5.0
BASELINE_WINDOW_START: float = -5.0
BASELINE_WINDOW_END: float = -2.0


def compute_segment_id(session_id: str, segment_window_start_utc: str, segment_window_end_utc:
↪  str) -> str:
    raw = f"{session_id}|{segment_window_start_utc}|{segment_window_end_utc}".encode("utf-8")
    return hashlib.sha256(raw).hexdigest()


def compute_soft_reward_candidate(p_match: Optional[float], proxy_reward: float) ->
↪  Optional[float]:
    if p_match is None:
        return None
    p = float(min(max(p_match, 0.0), 1.0))
    return p * float(proxy_reward)


def compute_au12_attribution_metrics(au12_series: list[dict], stimulus_time_s: float) ->
↪  dict[str, Optional[float]]:
    baseline_obs = [
        o["intensity"] for o in au12_series
        if stimulus_time_s + BASELINE_WINDOW_START ≤ o["timestamp_s"] ≤ stimulus_time_s +
        ↪  BASELINE_WINDOW_END
    ]
    measure_obs = [
        o for o in au12_series
        if stimulus_time_s + STIMULUS_WINDOW_START ≤ o["timestamp_s"] ≤ stimulus_time_s +
        ↪  STIMULUS_WINDOW_END
    ]
    if not measure_obs:
        return {
            "au12_baseline_pre": float(np.mean(baseline_obs)) if baseline_obs else None,
            "au12_lift_p90": None,
```

```
            "au12_lift_peak": None,
            "au12_peak_latency_ms": None,
        }

    baseline_pre = float(np.mean(baseline_obs)) if baseline_obs else 0.0
    values = np.array([o["intensity"] for o in measure_obs], dtype=float)
    p90_post = float(np.percentile(values, 90))
    peak_idx = int(np.argmax(values))
    peak_post = float(values[peak_idx])
    peak_latency_ms = float((measure_obs[peak_idx]["timestamp_s"] - stimulus_time_s) * 1000.0)

    return {
        "au12_baseline_pre": baseline_pre,
        "au12_lift_p90": max(0.0, p90_post - baseline_pre),
        "au12_lift_peak": max(0.0, peak_post - baseline_pre),
        "au12_peak_latency_ms": peak_latency_ms,
    }


def compute_peak_lagged_corr(x: np.ndarray, y: np.ndarray, max_lag: int) ->
↪  tuple[Optional[float], Optional[int]]:
    best_r: Optional[float] = None
    best_lag: Optional[int] = None
    for lag in range(0, max_lag + 1):
        if lag == 0:
            x_lag = x
            y_lag = y
        else:
            x_lag = x[:-lag]
            y_lag = y[lag:]
        if len(x_lag) < 4 or len(y_lag) < 4:
            continue
        if np.std(x_lag) == 0.0 or np.std(y_lag) == 0.0:
            continue
        r, _ = pearsonr(x_lag, y_lag)
        if best_r is None or float(r) > best_r:
            best_r = float(r)
            best_lag = lag
    return best_r, best_lag
```

SECTION 8

Deterministic semantic scorer specification

Module D must determine whether a spoken utterance satisfies a predefined semantic rule such as a required greeting phrase. Traditional string-matching algorithms like Levenshtein distance or BLEU fail because human speech contains fillers, paraphrasing, and transcription noise.

To perform robust evaluation while maintaining reproducibility, LSIE-MLF uses a deterministic semantic scorer registry. In v3.4 the primary method is a local Cross-Encoder executed in the ML

Worker. An optional Azure OpenAI gray-band fallback may be invoked only when the calibrated primary score falls inside a configured ambiguity interval.

The `is_match` boolean remains the reward gate $G_t \in \{0, 1\}$. When `is_match` is True, P90 AU12 intensity contributes to the posterior update; otherwise the reward is zero. `confidence_score` is the active method's semantic probability estimate and remains analytical only in v3.4; it does not modulate the §7B reward path.

The `reasoning` field is a bounded semantic reason code, not a free-form rationale. Implementations must not persist free-form semantic rationales.

Semantic shadow mode runs a candidate semantic method in parallel with the active method and records its score for evaluation only. Shadow-mode outputs must not alter live `is_match`, `confidence_score`, reward computation, or Thompson Sampling updates.

8.1 Inference parameter constraints

PARAMETER	VALUE	JUSTIFICATION
`temperature`	`0.0`	Collapses the probability distribution to deterministic token selection.
`top_p`	`1.0`	Disables nucleus sampling ensuring deterministic token selection.
`max_tokens`	`500`	Ensures adequate space for gray-band fallback deliberation plus structured JSON output.
`response_format`	`JSON Schema Structured Outputs`	Forces the model to return machine-parseable JSON rather than conversational text.
`seed`	`42`	Activates Azure OpenAI deterministic fallback mode.
`cross_encoder_model_id`	`local://models/semantic/lsie-greeting-cross-encoder-v1`	Pins the primary local semantic scorer artifact used for greeting-rule matching across deployments.
`cross_encoder_model_version`	`lsie-greeting-cross-encoder-v1.0.0`	Pins the exact primary scorer release used for deterministic replay and audit.
`match_threshold`	`0.72`	Defines the deterministic probability cutoff: scores >= 0.72 are matches outside the gray band.
`gray_band_interval`	`0.58 ⩽ score < 0.72`	Defines the only score interval in which llm_gray_band fallback may execute; score = 0.72 is reserved for the direct match threshold and is outside the gray band.
`cross_encoder_device_mode`	`cuda:0 deterministic when GPU is available; cpu deterministic fallback only when semantic_device_mode=cpu is configured`	Pins the execution device policy so primary scorer behavior is reproducible across deployments.

PARAMETER	VALUE	JUSTIFICATION
semantic_-calibration_-version	semantic-greeting-calibration-v1.0.0	Pins the calibration mapping from raw model scores to semantic probability estimates.

8.2 Deterministic semantic scorer registry

METHOD	ROLE / MODE	DETERMINIS-TIC	DESCRIPTION
cross_encoder	primary local	Yes	Primary local deterministic scorer using local://models/semantic/lsie-greeting-cross-encoder-v1 version lsie-greeting-cross-encoder-v1.0.0 with semantic-greeting-calibration-v1.0.0.
llm_gray_band	fallback azure_openai	Yes	Deterministic Azure OpenAI fallback invoked only when the calibrated cross_encoder score is within 0.58 <= score < 0.72 and gray-band fallback is enabled.
azure_llm_legacy	legacy azure_openai	Yes	Retired legacy direct Azure OpenAI semantic scorer retained only for historical attribution records.

8.3 Expected output schema

greeting_eval.schema.json JSON

```
{
 "type": "object",
 "required": [
  "reasoning",
  "is_match",
  "confidence_score"
 ],
 "properties": {
  "reasoning": {
   "type": "string",
   "enum": [
    "cross_encoder_high_match",
    "cross_encoder_high_nonmatch",
    "gray_band_llm_match",
    "gray_band_llm_nonmatch",
    "semantic_local_failure_fallback",
    "semantic_timeout",
    "semantic_error"
   ],
   "description": "Bounded semantic reason code; not a free-form rationale."
  },
  "is_match": {
   "type": "boolean"
  },
```

```
  "confidence_score": {
   "type": "number",
   "minimum": 0.0,
   "maximum": 1.0
  }
 },
 "additionalProperties": false
}
```

8.4 System prompt (canonical)

system_prompt.txt TEXT

```
You are a deterministic semantic evaluation fallback.
Your only task is to resolve ambiguous utterances that were routed into the gray band by the
↪  primary local semantic scorer.

Inputs:
1. Expected Greeting Rule
2. Actual Utterance

Evaluation constraints:
- Minor transcription errors and filler words are acceptable if the semantic intent remains
↪  unchanged.
- Major deviations in subject matter must be rejected.
- Do not produce free-form explanations.

Return strict JSON with exactly these fields:
1. reasoning: bounded reason code, one of ["gray_band_llm_match", "gray_band_llm_nonmatch"]
2. is_match: boolean
3. confidence_score: number between 0.00 and 1.00

Return the result strictly as valid JSON without markdown formatting.
```

SECTION 9

Docker compose runtime topology

The LSIE-MLF runtime environment consists of multiple containerized services orchestrated through Docker Compose. Containers share a common bridge network and specific named volumes enabling high-throughput IPC communication and persistent storage where required. Docker Compose generation must render any container-level `command` array present in this topology. The Redis service therefore renders `redis-server --appendonly yes`, not merely a prose-only AOF requirement.

9.1 Container topology

CONTAINER	ROLE	IMAGE	CONFIGURATION
`api`	API Server providing FastAPI REST endpoints and physiological webhook ingress for Oura webhooks.	`python:3.11-slim`	Port: 8000:8000 Net: appnetwork GPU: None Restart: unless-stopped
`worker`	ML Worker executing Celery inference tasks.	`nvidia/cuda:12.2.2-cudnn8-runtime-ubuntu22.04`	Port: None Net: appnetwork GPU: All (capabilities: [gpu], driver: nvidia) Restart: on-failure:5
`redis`	Message Broker (Redis) used by Celery; runtime command must enable append-only persistence with `redis-server --appendonly yes`.	`redis:7-alpine`	Port: None Net: appnetwork GPU: None Restart: unless-stopped
`postgres`	Persistent Store (PostgreSQL) for experiment and metric data; internal service port 5432 is available only on the Docker bridge network.	`postgres:16-alpine`	Port: None Net: appnetwork GPU: None Restart: unless-stopped
`stream_scrcpy`	Capture Container extracting low-latency raw PCM audio and H.264 encoded video streams.	`ubuntu:24.04 + scrcpy v3.3.4`	Port: None Net: appnetwork GPU: None Restart: on-failure:3
`orchestrator`	Module C orchestrator assembling 30-second segments, processing AU12 frames, and dispatching Celery tasks.	`nvidia/cuda:12.2.2-cudnn8-runtime-ubuntu22.04`	Port: None Net: appnetwork GPU: All (capabilities: [gpu], driver: nvidia) Restart: on-failure:5

9.2 Volume mounts

VOLUME	MOUNT PATH	CONTAINERS	PURPOSE
`ipc-share`	`/tmp/ipc/`	stream_scrcpy, worker, orchestrator	Shared volume containing IPC named pipes.
`data-raw`	`/data/raw/`	worker, orchestrator	Transient debug media buffers.
`data-interim`	`/data/interim/`	worker, orchestrator	Intermediate processing artifacts.
`data-processed`	`/data/processed/`	worker, orchestrator, api	Validated inference outputs and fallback storage.
`pg-data`	`/var/lib/postgresql/data/`	postgres	Persistent PostgreSQL database storage.

9.3 GPU allocation

The ML Worker and orchestrator containers receive GPU access through Docker's `deploy.resources` reservation syntax and require CUDA 12 with cuDNN 8 for CTranslate2 compatibility. cuDNN 8 preserves Pascal SM 6.1 dp4a compatibility required by the target hardware.

GPU allocation — YAML

```yaml
deploy:
  resources:
    reservations:
      devices:
        - driver: nvidia
          count: all
          capabilities: [gpu]
```

9.4 Device passthrough

USB passthrough allows the capture container to communicate directly with the Android device through the host kernel device subsystem.

Device passthrough — YAML

```yaml
devices:
  - /dev/bus/usb:/dev/bus/usb
```

9.5 Network configuration

All services run within the same Docker bridge network allowing container name DNS resolution. Only the API Server exposes ports externally; Redis and PostgreSQL are reachable only by internal service name on the Docker bridge network.

9.6 Service dependency order

#	SERVICE	DEPENDS ON	READINESS GATE / NOTES
1	`redis`	—	redis-cli ping Starts first with append-only persistence enabled by redis-server –appendonly yes.
2	`postgres`	`redis`	pg_isready Starts after redis.
3	`stream_scrcpy`	`redis`	Starts after redis to initialize IPC pipe.
4	`worker`	`redis, postgres, stream_scrcpy`	Starts after redis, postgres, and stream_scrcpy.
5	`orchestrator`	`redis, postgres, stream_scrcpy`	Starts alongside worker after redis, postgres, and stream_scrcpy.
6	`api`	`redis, postgres`	Starts after redis and postgres.

SECTION 10

Dependency and runtime matrix

10.1 System requirements

COMPONENT	REQUIRED VER	NOTES
Python	3.11.x	All containers must run Python 3.11. Python 3.12+ is untested with CTranslate2.
NVIDIA CUDA	⩾12.0	Required for faster-whisper and CTranslate2 runtime compatibility.
cuDNN	⩾8.0	Bundled within nvidia/cuda:12.2.2-cudnn8-runtime-ubuntu22.04; cuDNN 8 preserves Pascal SM 6.1 dp4a compatibility required by the hardening baseline.
NVIDIA Container Toolkit	latest	Required for Docker GPU passthrough.
GPU VRAM	⩾6GB	Minimum VRAM for faster-whisper large-v3 INT8 plus MediaPipe runtime.
Docker Compose	v2.x	Required for deploy.resources GPU reservation syntax.
scrcpy	3.3.4	Required for raw Android USB audio/video extraction; v3.3.4 provides the –no-playback flag and dual-instance –port range support used by the capture entrypoint.

10.2 Python package pinning

PACKAGE	VERSION	CONTAINER(S)	PURPOSE
faster-whisper	1.2.1	worker	GPU accelerated speech transcription
mediapipe	0.10.9	worker image (orchestrator service uses same image)	Face Mesh 478 landmark inference for Module C AU12 extraction
praat-parselmouth	0.4.4	worker	Praat acoustic feature extraction
spacy	3.7.x	worker	NLP preprocessing pipeline
psycopg2-binary	2.9.x	worker, api	PostgreSQL driver
pandas	2.2.x	worker	Time-series data processing
scipy	1.13.*	worker	Statistical computation
celery	5.4.x	worker, api	Distributed task queue
redis	5.x	worker, api	Celery broker transport
fastapi	0.110.x	api	REST API framework
uvicorn	0.29.x	api	ASGI server runtime
pydantic	2.x	worker, api	Schema validation

PACKAGE	VERSION	CONTAINER(S)	PURPOSE
numpy	1.26.x	worker image (worker and orchestrator services)	Vectorized AU12 calculations and ML preprocessing
pycryptodome	⩾3.20.0	worker	AES-256-GCM encryption
patchright	1.58.2	worker	Stealth browser automation
TikTokLive	5.0.x	worker	TikTok WebSocket ingestion
PySide6	⩾6.11	N/A (host-installed; not containerized)	Native Qt framework for scripts/debug_studio.py and services/operator_console/
torch	2.4.x	worker	Local Cross-Encoder semantic scoring runtime
transformers	4.46.x	worker	Transformer runtime for deterministic local semantic scorer
sentence-transformers	3.2.x	worker	Cross-Encoder semantic similarity models

SECTION 11

Variable extraction matrix

Each analytical variable produced within LSIE-MLF is traceable to its source module, data type, storage location, and extraction methodology.

VARIABLE PROFILE	EXTRACTION & ROUTING
Encoded Video Stream byte stream (H.264 in MKV container) Module A → Transient (kernel buffer)	scrcpy USB mirroring; H.264 encoded stream in MKV container via /dev/bus/usb passthrough.
Raw Audio PCM byte stream (s16le) Unit/range: 48 kHz mono Module A → Transient IPC pipe buffer	scrcpy raw audio capture; s16le 48 kHz mono to IPC Pipe.
Live Event Payloads dict Module B → Transient deque buffer	TikTokLive WebSocket ingestion; parsed protobuf event dictionaries.
Action Combo Trigger bool Unit/range: True/False Module B → Persistent Store events table	Composite temporal event correlation across concurrent chat actions.

VARIABLE PROFILE	EXTRACTION & ROUTING
UTC Drift Offset `float` Unit/range: seconds Module C → Transient variable	ADB epoch polling every 30s; `drift_offset` = host UTC − Android epoch.
Resampled Audio Chunks `byte stream (s16le)` Unit/range: 16 kHz mono Module C → Transient stdout pipe	FFmpeg resampling subprocess; 48 kHz → 16 kHz mono s16le on `pipe:1`.
3D Facial Landmarks `np.ndarray (478,3)` Unit/range: normalized coordinates Module C → Transient memory	MediaPipe Face Mesh in Module C; 478-vertex 3D landmark array per sampled frame.
AU12 Intensity Score `float` Unit/range: 0.0–5.0 (compute_intensity) or 0.0–1.0 (compute_bounded_intensity for Thompson Sampling reward pipeline) Module C → Persistent Store metrics table	AU12Normalizer (§7A) executed in Module C; IOD-normalized lip-corner distance with neutral baseline subtraction. Exposed as 0.0–5.0 (legacy diagnostic) and 0.0–1.0 (bounded, for reward pipeline).
Vocal Pitch F0 `float` Unit/range: 75–600 Hz Module D → Persistent Store metrics table	Legacy segment-level compatibility variable (deprecated as a canonical acoustic output in v3.3). Praat via parselmouth; fundamental frequency extraction from speech audio. Downstream consumers must migrate to the windowed §7D F0 fields.
Jitter `float` Unit/range: 0–1 ratio Module D → Persistent Store metrics table	Legacy segment-level compatibility variable (deprecated as a canonical acoustic output in v3.3). Praat via parselmouth; cycle-to-cycle pitch perturbation ratio. Downstream consumers must migrate to the windowed §7D jitter fields.
Shimmer `float` Unit/range: 0–1 ratio Module D → Persistent Store metrics table	Legacy segment-level compatibility variable (deprecated as a canonical acoustic output in v3.3). Praat via parselmouth; cycle-to-cycle amplitude perturbation ratio. Downstream consumers must migrate to the windowed §7D shimmer fields.
ASR Transcription `string` Unit/range: UTF-8 text Module D → Persistent Store transcripts table	faster-whisper with CTranslate2 INT8 backend; segment-level speech-to-text.
Semantic Match `JSON object` Unit/range: confidence 0–1 Module D → Persistent Store evaluation table	Deterministic semantic scorer (§8); primary local Cross-Encoder with optional deterministic llm_gray_band fallback. Output contract contains bounded `reasoning` reason code, `is_match`, and `confidence_score`.

VARIABLE PROFILE	EXTRACTION & ROUTING
Evaluation Variance `float` Unit/range: 0–infinity Module E → Persistent Store experiments table	Thompson Sampling posterior statistics; variance of Beta distribution per arm.
External Context Metadata `JSON dict` Module F → Persistent Store context table	patchright headless browser scraping; creator and trending topic metadata.
gated_reward `float` Unit/range: [0.0, 1.0] Module E → Phase3_EncounterLog JSON payload	`compute_reward()` (§7B); r_t = `P90` × `semantic_gate`.
p90_intensity `float` Unit/range: [0.0, 1.0] Module E → Phase3_EncounterLog JSON payload	`compute_reward()` (§7B); 90th percentile of bounded AU12 in stimulus window.
semantic_gate `int` Unit/range: {0, 1} Module E → Phase3_EncounterLog JSON payload	`compute_reward()` (§7B); binary gate from `is_match`: 1 if True, 0 otherwise.
n_frames_in_window `int` Unit/range: [0, +infinity) Module E → Phase3_EncounterLog JSON payload	`compute_reward()` (§7B); count of AU12 observations in stimulus window.
au12_baseline_pre `float | null` Unit/range: [0.0, +infinity) or null Module E → Phase3_EncounterLog JSON payload and Persistent Store attribution_score table	`compute_reward()` (§7B) and §7E attribution diagnostics; mean bounded AU12 in the pre-stimulus window, or null.
stimulus_time `float` Unit/range: UTC epoch seconds Module C → Transient variable	Drift-corrected UTC epoch of stimulus injection; defines baseline and measurement windows for `compute_reward()` (§7B).
RMSSD (Streamer) `float | null` Unit/range: milliseconds, >= 0 or null Module C → Persistent Store physiology_log table	Derived at segment assembly from normalized physiological chunks buffered in Module C (§4.B.2, §4.C.4). Uses a trailing 5-minute window with ibi precedence over session. Forced to null when `validity_ratio < PHYSIO_VALIDITY_MIN`.

VARIABLE PROFILE	EXTRACTION & ROUTING
RMSSD (Operator) `float | null` Unit/range: milliseconds, >= 0 or null Module C → Persistent Store physiology_log table	Derived at segment assembly from normalized physiological chunks buffered in Module C (§4.B.2, §4.C.4). Uses a trailing 5-minute window with ibi precedence over session. Forced to null when `validity_ratio < PHYSIO_VALIDITY_MIN`.
Heart Rate (Streamer/Operator) `int | null` Unit/range: 20–300 BPM or null Module C → Persistent Store physiology_log table	Rounded mean of overlapping `heart_rate_items_bpm` values in the trailing derivation window (§4.C.4); null when no heart-rate values exist.
Physiological Validity Ratio `float` Unit/range: [0.0, 1.0] Module C → Persistent Store physiology_log table	`validity_ratio = sum(valid_sample_count) / max(sum(expected_sample_count), 1)` over the trailing derivation window (§4.C.4).
Physiological Validity Flag `bool` Unit/range: True / False Module C → Persistent Store physiology_log table	`is_valid = (validity_ratio ⩾ PHYSIO_VALIDITY_MIN)`; default threshold 0.80 (§4.C.4).
Physiological Source Kind `string` Unit/range: {'ibi', 'session'} Module C → Persistent Store physiology_log table	ibi when the snapshot is derived from interbeat-interval chunks; otherwise session when derived from provider-session HRV chunks (§4.C.4).
Physiological Derivation Method `string` Unit/range: {'provider', 'server'} Module C → Persistent Store physiology_log table	`server` for IBI-derived RMSSD; `provider` for provider-session HRV snapshots unless later recomputed (§4.C.4).
Physiological Window Length `int` Unit/range: seconds, > 0 Module C → Persistent Store physiology_log table	Length of the trailing derivation window; default `PHYSIO_DERIVE_WINDOW_S = 300` (§4.C.4).
Physiological Freshness `float` Unit/range: seconds, >= 0 Module C → Persistent Store physiology_log table	Computed at segment assembly: `wall_clock - source_timestamp_utc` (§4.C.4).
Physiological Staleness Flag `bool` Unit/range: True / False Module C → Persistent Store physiology_log table	`freshness_s > PHYSIO_STALENESS_THRESHOLD_S` (default 600s) (§4.C.4).

VARIABLE PROFILE	EXTRACTION & ROUTING
Co-Modulation Index `float | null` Unit/range: [-1.0, +1.0] or null Module E → Persistent Store co-modulation_log table	Pearson correlation on 1-minute resampled, time-aligned valid streamer/operator RMSSD sequences over a 10-minute rolling window (§7C).
Physiological Coverage Ratio `float` Unit/range: [0.0, 1.0] Module E → Persistent Store co-modulation_log table	Fraction of 1-minute analysis bins with valid paired RMSSD observations over the 10-minute co-modulation window (§7C).
F0 Validity (Measure Window) `bool` Unit/range: True / False Module D → Persistent Store metrics table	Observational acoustic validity gating (§7D); true only when W_measure satisfies voiced-coverage and stable-island thresholds.
F0 Validity (Baseline Window) `bool` Unit/range: True / False Module D → Persistent Store metrics table	Observational acoustic validity gating (§7D); true only when W_baseline satisfies voiced-coverage and stable-island thresholds.
Perturbation Validity (Measure Window) `bool` Unit/range: True / False Module D → Persistent Store metrics table	Perturbation-span validity gating (§7D); true only when W_measure contains an eligible perturbation span.
Perturbation Validity (Baseline Window) `bool` Unit/range: True / False Module D → Persistent Store metrics table	Perturbation-span validity gating (§7D); true only when W_baseline contains an eligible perturbation span.
Voiced Coverage (Measure Window) `float` Unit/range: >= 0 seconds Module D → Persistent Store metrics table	Cumulative voiced coverage in W_measure computed under §7D voiced-only masking.
Voiced Coverage (Baseline Window) `float` Unit/range: >= 0 seconds Module D → Persistent Store metrics table	Cumulative voiced coverage in W_baseline computed under §7D voiced-only masking.
F0 Mean (Measure Window) `float | null` Unit/range: 75–600 Hz or null Module D → Persistent Store metrics table	Mean voiced-only F0 in W_measure under §7D; null when f0_valid_measure is false.

VARIABLE PROFILE	EXTRACTION & ROUTING
F0 Mean (Baseline Window) `float | null` Unit/range: 75–600 Hz or null Module D → Persistent Store metrics table	Mean voiced-only F0 in W_baseline under §7D; null when f0_valid_baseline is false.
F0 Delta (Semitones) `float | null` Unit/range: real-valued semitone difference or null Module D → Persistent Store metrics table	12 * log2(f0_mean_measure_hz / f0_mean_baseline_hz) under §7D; null when either dependent F0 mean is null or baseline F0 is non-positive.
Jitter Mean (Measure Window) `float | null` Unit/range: 0–1 ratio or null Module D → Persistent Store metrics table	Praat local jitter computed on the selected perturbation span in W_measure under §7D; null when perturbation_valid_measure is false.
Jitter Mean (Baseline Window) `float | null` Unit/range: 0–1 ratio or null Module D → Persistent Store metrics table	Praat local jitter computed on the selected perturbation span in W_baseline under §7D; null when perturbation_valid_baseline is false.
Jitter Delta `float | null` Unit/range: real-valued difference or null Module D → Persistent Store metrics table	jitter_mean_measure - jitter_mean_baseline under §7D; null when either dependent jitter mean is null.
Shimmer Mean (Measure Window) `float | null` Unit/range: 0–1 ratio or null Module D → Persistent Store metrics table	Praat local shimmer computed on the selected perturbation span in W_measure under §7D; null when perturbation_valid_measure is false.
Shimmer Mean (Baseline Window) `float | null` Unit/range: 0–1 ratio or null Module D → Persistent Store metrics table	Praat local shimmer computed on the selected perturbation span in W_baseline under §7D; null when perturbation_valid_baseline is false.
Shimmer Delta `float | null` Unit/range: real-valued difference or null Module D → Persistent Store metrics table	shimmer_mean_measure - shimmer_mean_baseline under §7D; null when either dependent shimmer mean is null.

VARIABLE PROFILE	EXTRACTION & ROUTING
Segment ID `string` Unit/range: 64-character lowercase hex Module C → InferenceHandoffPayload and Persistent Store attribution_event table	Deterministic SHA-256 over stable segment identity fields (`session_id`, `segment_window_start_utc`, `segment_window_end_utc`) in Module C (§6.1, §7E).
Bandit Decision Snapshot `JSON object` Module C → InferenceHandoffPayload and Persistent Store attribution_event table	Captured in Module C immediately after Thompson Sampling arm selection and before any posterior update; includes selected arm, candidate arms, posterior parameters, sampled thetas when available, and expected greeting.
semantic_p_match `float` Unit/range: [0.0, 1.0] Module D → Persistent Store evaluation table and attribution_score table	Deterministic semantic probability estimate produced by the active §8 semantic method; primary local Cross-Encoder in v3.4, optional gray-band fallback.
semantic_reason_code `string` Unit/range: bounded enum Module D → Persistent Store evaluation table	Bounded semantic route/result code emitted by the deterministic semantic scorer contract (§8).
soft_reward_candidate `float | null` Unit/range: [0.0, 1.0] or null Module E → Persistent Store attribution_score table	`soft_reward_candidate = semantic_p_match * proxy_reward` under §7E observational attribution only.
au12_lift_p90 `float | null` Unit/range: [0.0, 1.0] or null Module E → Persistent Store attribution_score table	`max(0, P90_post - au12_baseline_pre)` under §7E.
au12_lift_peak `float | null` Unit/range: [0.0, 1.0] or null Module E → Persistent Store attribution_score table	`max(0, Pmax_post - au12_baseline_pre)` under §7E.
au12_peak_latency_ms `float | null` Unit/range: milliseconds or null Module E → Persistent Store attribution_score table	Latency from stimulus timestamp to the peak AU12 observation in the post-stimulus measurement window under §7E.
sync_peak_corr `float | null` Unit/range: [-1.0, 1.0] or null Module E → Persistent Store attribution_score table	Maximum lagged Pearson correlation over the configured lag range under §7E lag-aware synchrony analytics.

VARIABLE PROFILE	EXTRACTION & ROUTING
sync_peak_lag `int | null` Unit/range: lag units or null Module E → Persistent Store attribution_score table	Lag value associated with `sync_peak_corr` under §7E lag-aware synchrony analytics.
outcome_link_lag_s `float` Unit/range: seconds, >= 0 Module E → Persistent Store event_outcome_link table	Deterministic event→outcome lag in seconds stored on EventOutcomeLink records under §7E.
attribution_finality `string` Unit/range: {'online_provisional', 'offline_final'} Module E → Persistent Store attribution tables	Lifecycle state assigned by Module E attribution finalization jobs under §7E.

SECTION 12

Error handling and recovery specification

LSIE-MLF defines four standardized failure categories. Each module must implement deterministic recovery behavior for each category. Physiological webhook rejections such as invalid HMAC signatures, malformed payloads, duplicate suppression, and Redis enqueue retries are application-level guardrails, not separate failure categories. Stale physiological snapshots are annotated with `is_stale = true`; they do not block segment dispatch. Module E excludes stale snapshots from co-modulation computation. Acoustic insufficiency is also a data-quality condition: when voiced coverage is insufficient, perturbation spans are invalid, or local Praat extraction fails, Module D records deterministic false validity flags and null dependent acoustic outputs without retrying the task, crashing the worker, or blocking downstream persistence.

12.1 Network disconnection

MODULE	BEHAVIOR
Module A	N/A
Module B	Exponential WebSocket reconnection up to 10 retries; physiological webhook ingress is handled by the API Server, and upstream transport failures rely on provider-managed retries after timeout or HTTP 503 responses
Module C	N/A
Module D	Network disconnection affects only `llm_gray_band` fallback. Retry the Azure OpenAI gray-band request once, then emit a deterministic semantic error reason code and bounded false/null-compatible outputs according to §8 while local cross-encoder primary scoring remains available.

MODULE	BEHAVIOR
Module E	Buffer database writes and retry every 5s
Module F	Return empty scrape result

12.2 Hardware device loss

MODULE	BEHAVIOR
Module A	USB reconnection polling every 2s for 60s then container restart
Module B	N/A
Module C	Freeze drift offset after 3 consecutive ADB poll failures (`DRIFT_FREEZE_AFTER_FAILURES` = 3). While frozen, return the cached offset without additional ADB calls; reset drift to zero after 300 seconds / 5 minutes of frozen state (`DRIFT_RESET_TIMEOUT` = 300).
Module D	Terminate worker container if GPU unavailable
Module E	N/A
Module F	N/A

12.3 Worker process crash

MODULE	BEHAVIOR
Module A	Pipe overflow handled via EAGAIN non-blocking writes
Module B	Reconnect WebSocket on restart
Module C	Restart FFmpeg within 1 second when AudioResampler detects a dead subprocess or EOF. If the video capture thread dies after an IPC / PyAV pipe break, the run loop recreates and restarts VideoCapture so the upstream scrcpy path can emit a fresh MKV header.
Module D	Docker restart policy on-failure:5
Module E	Log Celery task failure and continue
Module F	Retry scraping task up to two times

12.4 Queue overload

MODULE	BEHAVIOR
Module A	Kernel pipe overflow discards data silently
Module B	Deque ring buffer evicts oldest entries; physiological event enqueue uses Redis list `physio:events`, and enqueue backpressure returns HTTP 503 so the provider retries instead of silently dropping events
Module C	Deque maxlen eviction policy; bounded physiological drain (max 100 events per cycle) prevents stale or bursty webhook traffic from stalling 30-second segment assembly, and malformed physiological events are logged and discarded
Module D	Celery queue buffers tasks in Redis

MODULE	BEHAVIOR
Module E	Overflow records written to CSV fallback storage; when physiological coverage is incomplete or non-stale paired samples are insufficient, Module E records a null co-modulation result rather than failing the analytics pipeline
Module F	Limit concurrent scraping tasks

SECTION 13

Autonomous implementation audit checklist

Before deployment the system must satisfy every verification item defined below to ensure deterministic implementation.

#	AUDIT ITEM	VERIFICATION CRITERION
1	Directory structure	Repository directories match the hierarchy defined in Section 3.
2	Docker topology	Six containers are defined with correct images, networks, volumes, and restart policies.
3	IPC lifecycle	Named pipe lifecycle implemented according to the Module A IPC lifecycle steps in §4.A.
4	Audio pipeline	FFmpeg resampling from 48 kHz to 16 kHz executed correctly.
5	Drift correction	ADB polling every 30 seconds with `drift_offset` applied.
6	AU12 implementation	Landmark indices and epsilon guard implemented correctly.
7	Semantic scorer determinism	The configured primary semantic method produces deterministic outputs for identical inputs. When `llm_gray_band` is enabled, Azure OpenAI fallback remains configured with temperature 0, top_p 1, seed 42, and structured outputs according to §8.
8	Ephemeral Vault	AES-256-GCM encryption and 24-hour secure deletion enforced.
9	Schema validation	InferenceHandoffPayload validated via Pydantic.
10	Module contracts	Inputs, outputs, dependencies, side effects, and failures implemented exactly as specified.
11	Error handling	All four failure categories implemented across modules.
12	Dependency versions	Pinned runtime versions match the dependency matrix.
13	Variable traceability	All variables defined in the extraction matrix are produced by the correct module.
14	Data classification	Transient, debug, and permanent storage tiers enforced.
15	Canonical terminology	Only canonical component names are used throughout code and configuration.

#	AUDIT ITEM	VERIFICATION CRITERION
16	Reward pipeline	Thompson Sampling reward pipeline implements the mathematical recipe from §7B: P90 aggregation, binary semantic gate, fractional Beta-Bernoulli posterior updates with Beta(1,1) prior, and stimulus window boundaries [t+0.5s, t+5.0s].
17	Physiological schema compatibility	`_physiological_context` is optional on InferenceHandoffPayload. Payloads without physiological data remain valid. The legacy JSON field `physiological_sample_event_schema` contains the canonical PhysiologicalChunkEvent definition.
18	Physiological freshness gating	is_stale is True when freshness_s exceeds PHYSIO_STALENESS_-THRESHOLD_S. Stale samples do not block segment dispatch. Module E excludes stale samples from co-modulation computation.
19	Physiological data governance	No raw Oura webhook JSON payloads reach the Persistent Store. Only normalized scalar derivatives and metadata (RMSSD, HR, validity_ratio, freshness, source_kind, derivation_method, co-modulation index) are persisted to physiology_log and comodulation_log tables.
20	Co-modulation determinism	Co-modulation analytics produce null when fewer than 4 paired 1-minute aligned observations exist or either aligned series has zero variance. Pearson correlation uses scipy.stats.pearsonr. Results are deterministic for the same input data.
21	Reward pipeline invariance	`compute_reward()` and `ThompsonSamplingEngine.update()` follow §7B exactly. Physiological co-modulation (§7C), observational acoustic analytics (§7D), semantic probability scores beyond the binary gate (§8), and attribution analytics (§7E) are observational only under the §7B reward-path invariance rule.
22	Observational acoustic validity contract	Windows that fail §7D voiced-coverage or perturbation constraints emit deterministic false validity flags and null dependent acoustic fields.
23	Observational acoustic payload/storage conformance	Module D → Module E payload and Persistent Store schema include the canonical §7D acoustic fields, and no raw audio or reconstructive voiceprint data is persisted.
24	Attribution ledger schema conformance	AttributionEvent, OutcomeEvent, EventOutcomeLink, and AttributionScore validate against §6.4 and persist with required version/finality fields.
25	Stable segment identity	The same payload replayed with identical segment window bounds produces the same `segment_id` deterministically.
26	Bandit decision snapshot capture	BanditDecisionSnapshot is captured in Module C at arm-selection time before posterior update and propagated to Module E attribution persistence.
27	Semantic scorer shadow/primary separation	When semantic shadow mode (defined in §8) is enabled, shadow scores do not alter live `is_match`, `confidence_score`, or §7B reward behavior. When promoted, gray-band fallback executes only within the configured interval 0.58 <= score < 0.72, and deterministic match cutoff is 0.72.
28	Attribution backfill idempotency	Reprocessing the same time range does not create duplicate attribution events, outcomes, links, or scores.
29	Outcome linker determinism	Given identical events, outcomes, attribution horizon, and link_rule_version, EventOutcomeLink outputs including `lag_s` are deterministic.
30	Derived-only attribution persistence compliance	No raw audio, raw video, complete transient PhysiologicalChunkEvent payload bodies, or free-form semantic rationales are persisted by the attribution extension.

#	AUDIT ITEM	VERIFICATION CRITERION
31	Attribution reward-path invariance	Enabling §7E analytics produces zero difference in `compute_reward()`, RewardResult, Thompson Sampling posterior updates, and arm-selection logic for identical inputs in v3.4.

— END OF SPECIFICATION —